An

Internship Report

On

AI-Powered Code Reviewer and Quality Assistant

By

SHRIKRUSHNA UDAY GANDHEWAR

Under the Guidance

of

Ms. Mukta Shelke

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Mahatma Gandhi Mission's College of Engineering, Nanded (M.S.)

Academic Year 2025-26

An Internship Report on

"AI-Powered Code Reviewer and Quality Assistant"

Submitted to

DR. BABASAHEB AMBEDKAR TECHNOLOGICAL UNIVERSITY, LONERE.

for fulfilment of the requirement for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING

By

SHRIKRUSHNA UDAY GANDHEWAR

Under the Guidance

of

Ms. Mukta Shelke

(DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MAHATMA GANDHI MISSION'S COLLEGE OF ENGINEERING

NANDED(M.S.)

Academic Year 2025-26

CERTIFICATE

This is to certify that the internship entitled

"AI-Powered Code Reviewer and Quality Assistant"

Being submitted by ***MR. SHRIKRUSHNA UDAY GANDHEWAR*** *to the Dr. Babasaheb Ambedkar Technological University, Lonere, for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of bonafide work carried out by him under my supervision and guidance. The matter contained in this report has not been submitted to any other university or institute for the award of any degree*.

Ms. Mukta Shelke

Guide

Dr. A. M. Rajurkar

H.O.D

Computer Science & Engineering

Dr. G.S. Lathkar

Director

MGM's College of Engineering, Nanded.

INTERNSHIP OFFER LETTER

Infosys_Project AI-Powered Code Reviewer and Quality Assistant(8:20 PM – 9:20 PM)_Batch-13 _ Meeting Invite.

1 message

Ajay Kumar B <springboardmentor0001@gmail.com> Thu, Feb 26, 2026 at 10:37 AM
To: Ajay Kumar B <springboardmentor0001@gmail.com>
Bcc: shrikrushnagandhewar@gmail.com

Dear Associates,

Greetings from Infosys!

We are delighted to welcome you to the Infosys _Project AI-Powered Code Reviewer and Quality Assistant_Batch-13 _ Meeting Invite.

Batch Details
Training Start Date: 14th February 2026
Training Timings: 8:20 PM – 9:20 PM IST (Daily, 40 days)

Note:

You are requested to attend the upcoming 40-day training sessions by joining through the Microsoft Teams link provided below.

The sessions will commence on **Thursday, 26th February 2026**, and attendance is mandatory. Please ensure your consistent participation.

Meeting Link: https://teams.microsoft.com/meet/43501385297596?p=ZQe9wcRvUeJFpoG8r2

Don't miss this valuable opportunity to gain in-depth insights and hands-on experience with Infosys.

Regards,
Infosys Mentor

COMPLETION LETTER

5/15/26, 4:13 PM Gmail - Congratulations on successfully completing the Infosys Springboard Internship 6.0

Gmail

Shrikrushna Gandhewar <shrikrushnagandhewar@gmail.com>

Congratulations on successfully completing the Infosys Springboard Internship 6.0
1 message

Infosys Springboard-Support <Springboard-support@infosys.com> 29 April 2026 at 15:26

Dear Intern,

Congratulations! You have successfully completed Infosys Springboard Internship 6.0! We're proud of your achievement and the dedication you've shown throughout the internship.

Your certificate will be sent to your registered email address shortly. Keep up the great work and continue striving for excellence!

Announce your achievement and about your internship experience on social media. Tag us @InfosysSpringboard when you do.

Note: Your project is your asset. Refrain from sharing your artifacts in online forums/domains to safeguard your work's ownership

Regards,
Team Infosys Springboard

Keep your learning journey going. Scan the QR code to download the **Infosys Springboard App** and access your courses on the go!

App Store Play Store

Follow us on Facebook | Instagram | LinkedIn | X (Twitter) | Threads

In case of queries, write to springboard-support@infosys.com.

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my internship guide, **Ms. Mukta Shelke** for her invaluable support, guidance, and encouragement throughout the duration of this Internship. Her profound knowledge and expertise have been instrumental in the successful completion of this work. Her patience and willingness to assist me at every step have greatly enriched my learning experience. Her constructive feedback and insightful suggestions have not only helped me overcome challenges but also motivated me to strive for excellence.

I gladly take this opportunity to thank **Dr. A. M. Rajurkar** (Head of Computer Science & Engineering, MGM's College of Engineering, Nanded).

I am heartily thankful to **Dr. G. S. Lathkar** (Director, MGM's College of Engineering, Nanded) for providing facility during progress of project also for her kindly help, guidance and inspiration.

Last but not least I also thankful to all those who help directly or indirectly to develop this project and complete it successfully.

With Deep Reverence

SHRIKRUSHNA UDAY GANDHEWAR

(B-Tech CSE-II)

ABSTRACT

The AI-Powered Code Reviewer and Quality Assistant is an intelligent software analysis platform developed to improve code quality, documentation, and maintainability for both Python and Java projects. The system combines static code analysis techniques with Artificial Intelligence to automate important software engineering tasks such as documentation generation, validation, syntax checking, optimization, and software quality analysis. The project is designed to reduce manual effort during code review and help developers understand and maintain source code more efficiently.

The application processes uploaded source code files and extracts important programming information such as functions, classes, methods, parameters, return values, and control-flow structures. Python source code is analyzed using AST-based processing, while Java source code is handled using Regex-based analysis methods. Based on the extracted information, the system generates Python docstrings and Java Javadocs automatically using AI integration through Groq API and LangChain. The platform also performs documentation validation, complexity analysis, maintainability evaluation, and syntax error detection to improve overall software quality.

The system includes interactive dashboards and visual reports developed using Streamlit and Plotly to help users understand project metrics and analysis results more clearly. Features such as optimization suggestions, AI-generated explanations, auto-fix support, and JSON report generation make the application useful for developers, students, and software teams. By combining AI-based processing with software engineering concepts, the project provides an organized and efficient solution for intelligent code review and automated documentation management.

TABLE OF CONTENTS

LIST OF FIGURES

Chapter 1

INTRODUCTION

Infosys Springboard Virtual Internship 6.0 is a learning-focused internship program created to help students gain practical experience in different technology domains through real-world projects and industry-based training. The program provides an opportunity to improve technical skills, programming knowledge, and problem-solving abilities by working on hands-on development activities.

The internship in the Python domain helps students understand software development concepts using Python and related technologies. Participants work on practical projects that involve coding, application development, data handling, Artificial Intelligence concepts, and software analysis techniques. This experience helps students connect academic learning with real development practices used in the software industry.

The program also supports skill development in areas such as logical thinking, project implementation, teamwork, and technical communication. By working on project-based tasks, students gain better understanding of how software systems are planned, developed, tested, and maintained in professional environments.

The AI-Powered Code Reviewer and Quality Assistant project was developed during the Infosys Springboard Virtual Internship 6.0 under the Python domain. The project focuses on intelligent code analysis, automated documentation generation, validation, optimization, and reporting for both Python and Java source code using AI-based technologies and software engineering concepts.

1.1 Introduction to the Python Domain

Python is a popular programming language known for its simple syntax, readability, and ease of use. It is widely used in different areas of software development such as web applications, Artificial Intelligence, Machine Learning, automation, data analysis, cybersecurity, and software testing. Because of its flexibility and beginner-friendly structure, Python is commonly used by both students and professional developers for building a variety of applications.

One of the main strengths of Python is the availability of a large number of libraries and frameworks that simplify development tasks. Developers can use Python to create interactive applications, process data, connect with APIs, and build AI-based

systems with less complexity. Technologies such as Streamlit, Plotly, LangChain, and other Python libraries help in developing intelligent and user-friendly software solutions more efficiently.

Python also supports different programming styles, including procedural, object-oriented, and functional programming. This makes it suitable for creating scalable and maintainable software systems. Its clear syntax and organized structure help developers write understandable code and improve overall productivity during software development.

In this project, Python is used as the main development language for implementing source code analysis, AI integration, automated documentation generation, validation, optimization, and report generation features. The language provides the flexibility and technical support required to build an intelligent system capable of handling both Python and Java source code processing tasks effectively.

1.2 Internship Objectives

The primary objective of the internship was to gain practical knowledge of software development by working on a real-world project in the Python domain. The internship helped in understanding how modern applications are developed, analysed, and maintained using current technologies, programming practices, and AI-based tools.

Another important goal was to study the role of Artificial Intelligence in software engineering tasks such as code analysis, automated documentation generation, validation, optimization, and reporting. The project provided hands-on experience in building an intelligent system that can reduce manual effort and improve software quality through automation. The internship also focused on improving programming skills, logical thinking, and problem-solving abilities using Python and related technologies. Working on the project provided practical exposure to APIs, frameworks, libraries, dashboard development, and software analysis techniques used in real development environments.

In addition, the internship aimed to develop a better understanding of software maintainability, documentation standards, and intelligent automation methods. The overall objective was to create a system capable of analyzing both Python and Java source code while generating useful documentation, validation results, and software quality insights in an efficient and organized manner.

1.3 Scope of the Project

The AI-Powered Code Reviewer and Quality Assistant project is designed to support intelligent source code analysis, automated documentation generation, validation, optimization, and reporting for both Python and Java programs. The system aims to simplify software review and documentation tasks by reducing the amount of manual effort required during development and maintenance activities.

The project focuses on features such as generating Python docstrings and Java Javadocs, checking documentation quality, detecting syntax-related issues, analyzing code complexity, and providing optimization suggestions. It also includes AI-based assistance that helps users understand functions, methods, and overall program logic more easily.

The application supports analysis of multiple files and complete project folders, allowing users to work with larger codebases efficiently. Interactive dashboards and visual reports are included to display software quality metrics, maintainability information, and documentation coverage in a more understandable way. Technologies such as Python, Streamlit, Plotly, Groq API, LangChain, Radon, and pydocstyle are used to build the system and support intelligent processing operations.

The project is mainly developed for learning, software analysis, and development support purposes. It provides a foundation for future improvements such as support for additional programming languages, stronger AI capabilities, IDE integration, and more advanced software quality analysis features.

1.4 Internship Duration and Deliverables

The Infosys Springboard Virtual Internship 6.0 was completed over a scheduled internship period focused on learning, project development, and practical implementation in the Python domain. During the internship, different phases of the project were carried out, including requirement understanding, system planning, development, testing, documentation, and final project preparation. The internship provided practical experience in working with real-world software development concepts and helped improve technical, analytical, and problem-solving skills.

The major outcome of the internship was the development of the AI-Powered Code Reviewer and Quality Assistant. The system includes features such as Python docstring generation, Java Javadoc generation, source code validation, syntax error detection, AI-based function explanation, optimization suggestions, maintainability

analysis, and interactive dashboards for software quality evaluation. The project also involved preparing reports, diagrams, validation outputs, and presentation materials related to the developed system.

The internship helped in gaining practical knowledge of Artificial Intelligence integration, software quality analysis, backend processing, dashboard development, and automated documentation techniques. The completed project demonstrates how Python and AI technologies can be combined to create intelligent tools that support software development and maintenance activities more efficiently.

1.5 Introduction to AI-Powered Code Review Systems

AI-Powered Code Review Systems are software tools that use Artificial Intelligence to examine source code and assist developers in improving software quality. These systems help automate tasks that are normally performed manually during code review, such as checking documentation, identifying coding issues, analysing complexity, and suggesting improvements. By combining AI techniques with software analysis methods, these tools make the development process faster and more efficient.

In traditional software development, reviewing source code manually can take a significant amount of time, especially in large projects containing many files and functions. Developers need to inspect code structure, documentation quality, syntax, maintainability, and possible errors carefully. AI-based review systems help reduce this effort by analyzing the code automatically and generating useful insights and recommendations.

Modern AI-driven code review tools can perform tasks such as generating Python docstrings and Java Javadocs, detecting syntax-related problems, providing optimization suggestions, explaining functions, and evaluating software quality metrics. These systems can understand code structure and generate context-aware responses that help developers write cleaner, more understandable, and better-documented code.

The AI-Powered Code Reviewer and Quality Assistant project is developed using these concepts to support intelligent analysis for both Python and Java source code. The system combines AI processing, validation, metrics analysis, and visualization features to create a structured environment for improving software documentation, maintainability, and overall code quality.

1.6 Problem Statement

In modern software development, managing source code in a clean and well-documented manner becomes difficult as projects grow larger and more complex. During development, programmers usually concentrate on completing features and meeting project deadlines, while activities such as documentation, code review, and maintainability checking are often postponed or ignored. Because of this, many applications contain unclear logic, inconsistent coding practices, and insufficient documentation, which later creates problems during debugging, updating, or team collaboration.

Most existing code analysis tools mainly focus on basic operations such as syntax checking, formatting validation, or static analysis. Although these tools are useful for identifying technical errors, they are not capable of understanding the actual purpose of the code or generating meaningful explanations automatically. Developers still spend a considerable amount of time manually reviewing functions, writing docstrings, checking maintainability, and ensuring coding standards are properly followed. This process becomes even more difficult when working with large projects or unfamiliar codebases.

Another challenge faced by developers is the absence of a single platform that combines documentation generation, intelligent code understanding, validation, and quality analysis together. In many cases, separate tools are required for measuring complexity, checking documentation coverage, generating reports, and reviewing source code quality. Using multiple tools increases the workload and makes the development process less organized. In academic projects and collaborative environments, poorly documented code can also make it difficult for students, developers, or maintenance teams to understand the overall system properly.

To overcome these issues, the proposed AI-Powered Code Reviewer and Quality Assistant is developed as an automated solution that supports intelligent source code analysis and software quality improvement. The system uses Artificial Intelligence along with AST-based processing techniques to generate documentation, validate coding standards, analyze maintainability, and provide optimization suggestions. By combining these features into a single platform, the project helps reduce manual effort and makes software maintenance and code understanding more efficient.

1.7 Proposed Solution

The proposed solution for this project is an AI-Powered Code Reviewer and Quality Assistant that helps users analyze, document, validate, and improve Python and Java source code automatically. The system is developed to simplify software review activities by reducing the amount of manual work required during documentation and code quality analysis. It provides an organized platform where users can perform multiple software analysis tasks within a single application.

The application reads uploaded source code files and extracts important programming details such as functions, classes, methods, parameters, return values, and control-flow structures. Using this information, the system generates Python docstrings and Java Javadocs automatically with the help of AI-based processing. This helps developers maintain better documentation and improves overall code understanding.

The proposed system also checks software quality by performing validation, complexity analysis, maintainability evaluation, and syntax error detection. It provides optimization suggestions and code improvement recommendations that help users identify areas where the source code can be made cleaner, more efficient, and easier to maintain.

To make the analysis results easier to understand, the system includes interactive dashboards, visual reports, charts, and AI-generated explanations. These features help users review project quality, documentation coverage, and software metrics in a more organized and readable format. By combining Artificial Intelligence with software engineering techniques, the project provides an efficient solution for intelligent code review and automated software quality management for both Python and Java projects.

1.8 Report Scope

This report presents the design, implementation, and functionality of the AI-Powered Code Reviewer and Quality Assistant developed during the internship project. The main purpose of the system is to simplify tasks related to source code analysis, automated documentation, validation, and software quality evaluation using Artificial Intelligence techniques. The project is designed to reduce the amount of manual work involved in reviewing and maintaining source code while helping developers improve overall code quality and readability.

The report explains the architecture of the application, the role of different modules, and the technologies used during development. It describes how features such as AST-based source code processing, AI-generated docstrings, validation mechanisms, complexity analysis, and dashboard visualization are implemented within the system. Technologies and tools including Python, Streamlit, Groq API, LangChain, Radon, and pydocstyle are integrated to support intelligent analysis and reporting functionalities. In addition, the documentation describes the frontend workflow, report generation mechanism, testing procedures, and storage of generated analysis results. The report also explains how uploaded source code files move through different processing stages, beginning from file handling and ending with visualization and report generation. Screenshots, sample outputs, and workflow examples are included to demonstrate the practical working of the application.

The overall objective of the project is to improve software readability, maintainability, and documentation quality through automation and AI-assisted analysis. Although the application mainly focuses on Python source code processing, it also provides support for Java-related documentation and validation features. Future enhancements may include support for additional programming languages, improved optimization capabilities, and scalability improvements for larger real-world software projects.

Chapter 2

SYSTEM ARCHITECTURE

The AI-Powered Code Reviewer and Quality Assistant is designed using a modular and well-structured architecture that allows the system to perform multiple software analysis tasks efficiently. The architecture is organized in such a way that each component of the system handles a separate responsibility, including source code processing, documentation generation, validation, metrics calculation, visualization, and report management. This modular design improves the flexibility, maintainability, and scalability of the application, making it easier to upgrade and extend in the future with additional programming language support or advanced AI features. The complete workflow of the system begins from the user interface layer, where users interact with the application through a web-based dashboard developed using Streamlit. The interface allows users to upload source code files and select different analysis features such as docstring generation, validation, metrics calculation, or function explanation. The frontend is designed to be simple and interactive so that both beginners and professional developers can use the system without difficulty. Once the user uploads the files, the application forwards the source code to the backend processing modules for further analysis.

The backend processing layer forms the core part of the system architecture. In this layer, the source code is first analyzed using Abstract Syntax Tree (AST) parsing techniques. The AST parser converts the program into a tree-like structure that represents the internal organization of the code. Through this process, the system extracts important programming elements such as functions, classes, arguments, return statements, loops, conditional blocks, exceptions, and decorators. This structured information helps the system understand how the code is written and provides the foundation for further intelligent processing. By using AST-based analysis instead of simple text processing, the system can perform more accurate and meaningful analysis of the uploaded source code. After extracting the code structure, the information is passed to the AI-powered analysis modules. These modules use Groq API and LangChain integration to generate automatic docstrings, explain the functionality of methods, and provide suggestions for improving readability and maintainability. The AI model processes the extracted metadata and creates human-readable explanations in different documentation formats such as Google Style, NumPy Style, and

reStructuredText. This feature reduces the manual effort required for writing technical documentation and helps developers maintain proper documentation standards throughout the project lifecycle.

Along with documentation generation, the system also contains validation and metrics analysis modules that evaluate the overall quality of the source code. The validation module checks whether the documentation follows proper formatting standards and identifies missing or incomplete docstrings. Libraries such as pydocstyle are used to perform style validation and detect documentation-related issues. At the same time, the metrics calculation module measures software quality parameters including cyclomatic complexity, maintainability index, and documentation coverage. These metrics help developers identify highly complex functions, repetitive logic, and sections of code that may become difficult to maintain in the future.

The results generated by all analysis modules are then transferred to the visualization and reporting layer. This layer uses Plotly and Streamlit components to display charts, graphs, coverage reports, and quality metrics in an interactive format. Users can easily understand the condition of the codebase through visual analytics and summary reports. The dashboard provides both file-level and project-level analysis, enabling developers to track overall software quality more effectively. In addition, the system stores generated reports and analysis data in JSON format, allowing users to access previous reports and maintain a history of analysis results for future comparison and review.The architecture of the system ensures smooth communication between all components, including the frontend interface, processing modules, AI services, validation tools, and reporting mechanisms. Each module works independently while sharing processed information through a structured workflow. This design not only improves system performance but also simplifies debugging, testing, and future development. Overall, the architecture of the AI-Powered Code Reviewer and Quality Assistant provides a reliable and efficient framework for intelligent code analysis, automated documentation generation, and software quality improvement.

2.1 Architectural Design of the System

The architectural design of the AI-Powered Code Reviewer and Quality Assistant is developed to provide an organized, modular, and efficient environment for source code analysis and documentation generation. The system is structured in multiple layers where each component performs a specific task while interacting with other modules

to complete the overall workflow. This design approach improves maintainability, scalability, and flexibility of the application, allowing new features and technologies to be integrated easily in the future.

The architecture begins with the user interface layer, which is developed using Streamlit. This layer acts as the interaction point between the user and the system. Through the web interface, users can upload source code files, select analysis options, generate docstrings, validate documentation, and view analytics reports. The interface is designed to be simple and user-friendly so that both beginners and experienced developers can use the application without technical difficulty.

Once the source code is uploaded, the files are forwarded to the core processing module, which forms the central part of the system. In this stage, the application uses Abstract Syntax Tree (AST) parsing techniques to examine the structure of the source code. The parser extracts important programming elements such as functions, classes, parameters, loops, return statements, and exception handling blocks. This structured extraction helps the system understand the logic and organization of the code more accurately than simple text-based analysis methods.

After the code structure is processed, the extracted information is passed to the AI analysis module. This module integrates Groq API and LangChain to generate intelligent docstrings, explain functions, and provide suggestions related to readability and maintainability. The AI component analyzes the extracted metadata and converts it into human-readable documentation following standard documentation styles. This reduces the manual effort required for writing technical documentation and improves the consistency of project documentation.

The system also contains validation and metrics modules responsible for checking documentation quality and software maintainability. These modules use tools such as pydocstyle and Radon to identify missing documentation, measure code complexity, and evaluate maintainability indexes. The results generated by these modules help developers identify problematic areas in the source code and improve software quality more effectively.

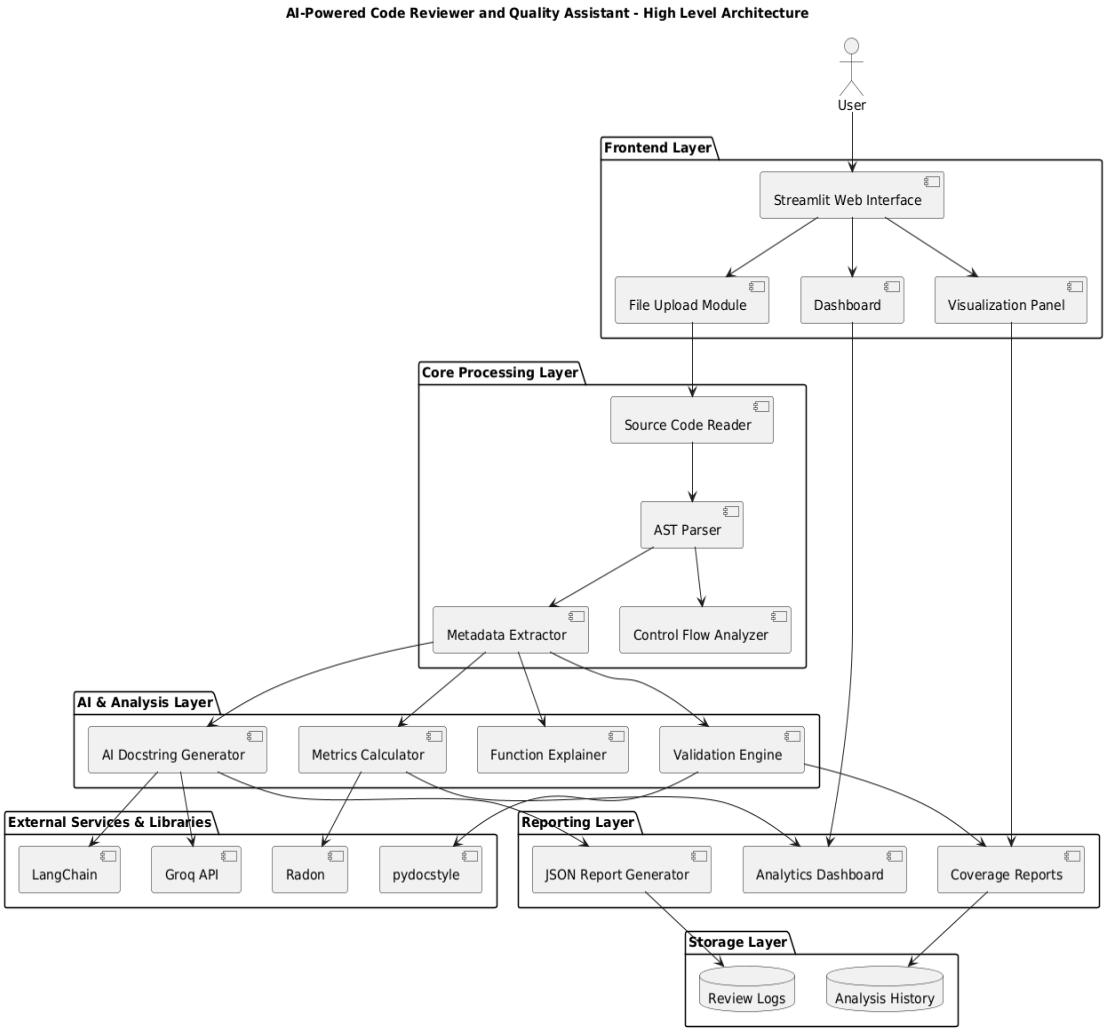


Figure 2.1 System Architecture

Finally, all generated reports and analysis results are displayed through the dashboard and visualization layer. Interactive charts and summary reports help users understand the overall condition of the project in a clear and graphical manner. The system also stores generated reports in JSON format for future reference and analysis tracking. Overall, the architecture of the proposed system provides a smooth workflow for automated code analysis, intelligent documentation generation, validation, and reporting while maintaining flexibility for future enhancements and scalability.

2.2 Major Components of the System

The AI-Powered Code Reviewer and Quality Assistant is designed using multiple interconnected modules, where each module handles a particular task within the system. Together, these components manage operations such as source code analysis, automated

documentation generation, software quality evaluation, and report visualization. The modular design keeps the application organized, makes maintenance easier, and provides flexibility for adding new features in the future.

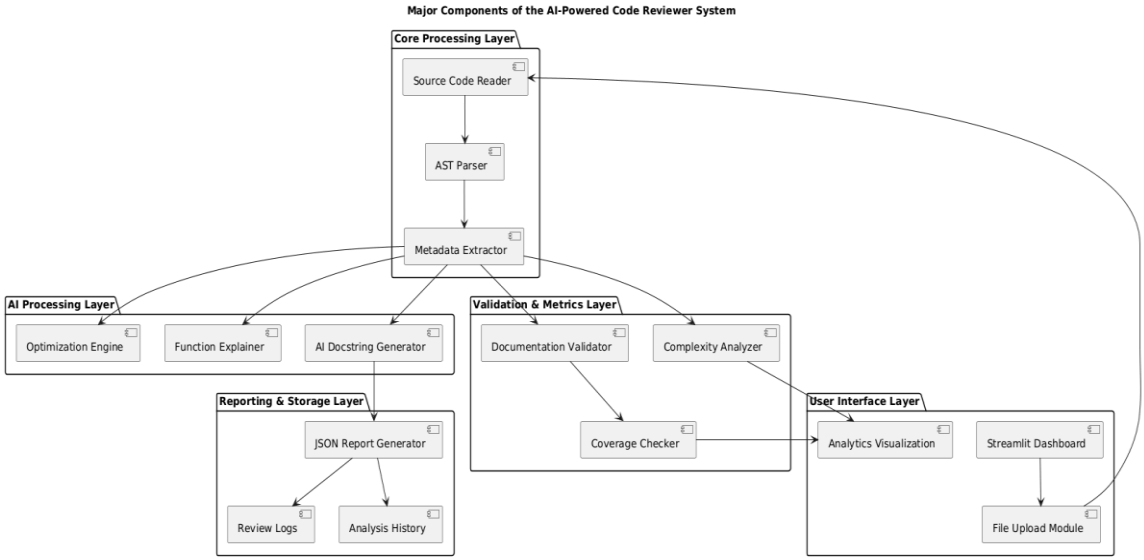


Fig.2.2 Major Components

One of the main parts of the system is the User Interface and Dashboard module, which is developed using Streamlit. This interface acts as the communication layer between the user and the application. Through the dashboard, users can upload Python or Java files, access analysis features, generate documentation, and review quality reports using interactive charts and visual summaries. The interface is kept simple and user-friendly so that both students and developers can navigate the system easily.

Another important part of the application is the Core Processing Module. This module is responsible for reading uploaded source code files and analyzing their structure. AST-based processing is used for Python files to identify functions, classes, loops, conditions, parameters, and exception-handling structures. The extracted information is converted into structured metadata, which is later shared with other modules for documentation generation, validation, and further analysis.

The system also contains an AI Analysis and Documentation Module that provides intelligent assistance using Groq API and LangChain integration. After receiving metadata from the processing module, the AI component generates docstrings, explains functions, and suggests possible code improvements. By automatically producing readable explanations and structured documentation, the module reduces manual documentation work and helps users understand the source code more clearly.

Another key section of the application is the Validation and Metrics Module. This module evaluates code quality by checking documentation standards, identifying missing docstrings, and calculating software metrics such as cyclomatic complexity and maintainability index. Libraries like pydocstyle and Radon are used during this process to help detect areas of the project that may require optimization or restructuring.

The Reporting and Storage Module manages the final analysis results produced by the system. It generates reports, displays summaries and graphical analytics, and stores processed data in JSON format for future reference. The stored records allow users to review previous analysis results, monitor improvements in project quality, and maintain a history of software review activities over time.

2.2.1 Core Processing Module

The Core Processing Module is one of the most important components of the AI-Powered Code Reviewer and Quality Assistant system. This module is responsible for reading uploaded source code files, analyzing their structure, and extracting important information required for further processing. It acts as the central processing unit of the application and provides the foundation for features such as documentation generation, code validation, metrics analysis, and function explanation. The module is designed to process source code in an organized and efficient manner while ensuring accurate extraction of programming elements.

The processing workflow begins with the Source Code Reader component, which accepts source code files uploaded by the user through the Streamlit interface. This component reads the contents of the uploaded files and prepares them for analysis. It ensures that the files are correctly loaded and transferred to the next stage of processing. The system mainly supports Python source files and handles the file content in a structured format to maintain consistency during analysis.

After reading the source code, the system performs AST Parsing using Python's Abstract Syntax Tree module. AST parsing converts the source code into a tree-like structure that represents the internal organization and logical flow of the program. Instead of treating the code as simple text, the parser understands programming constructs such as functions, classes, loops, conditions, exceptions, variables, and decorators. This method provides more accurate and meaningful analysis compared to basic text-processing techniques and forms the core of intelligent code understanding within the system.

Once the code structure is parsed, the Metadata Extraction process collects detailed information related to the identified programming elements. This includes function names, parameters, return values, decorators, docstrings, exception handling blocks, and other related details. The extracted metadata is organized into a structured format and forwarded to other modules such as AI documentation generation, validation, and metrics analysis. This process enables the system to generate meaningful explanations and perform software quality evaluation effectively.

The Core Processing Module also performs Control Flow Analysis to understand how the program executes through different logical paths. This analysis identifies loops, conditional statements, return blocks, yield statements, and exception handling structures present within the code. By studying the control flow, the system can evaluate the complexity and maintainability of the program more accurately. The collected information helps the application generate better AI explanations, identify complex code sections, and provide deeper insights into the overall behavior and structure of the software system.

2.2.2 AI Analysis and Documentation Module

The AI Analysis and Documentation Module is responsible for generating intelligent documentation, explaining source code functionality, and assisting developers with code understanding using Artificial Intelligence techniques. This module plays an important role in improving software readability and reducing the manual effort required for writing technical documentation. By combining code analysis with Large Language Models (LLMs), the system is capable of generating human-readable explanations and structured docstrings automatically.

One of the major functions of this module is AI Docstring Generation. After receiving extracted metadata from the Core Processing Module, the system analyzes functions, classes, parameters, and return statements to create professional documentation. The generated docstrings follow standard documentation formats such as Google Style, NumPy Style, and reStructuredText (reST). This feature helps developers maintain proper documentation standards while saving significant development time.

The module also includes a Function Explanation feature that helps users understand the logic and purpose of specific functions or code blocks. The system generates simplified explanations based on the extracted source code structure, making it easier for developers, students, and maintenance teams to understand complex

programs. This functionality is especially useful when working with unfamiliar codebases or large software projects.

Another important part of this module is Prompt Processing. Before sending requests to the AI model, the system prepares structured prompts using the extracted metadata and code information. These prompts help the AI model understand the context of the code more accurately and generate meaningful responses. Proper prompt construction improves the quality, clarity, and consistency of the generated documentation and explanations.

The module uses Groq API and LangChain Integration to communicate with Large Language Models efficiently. LangChain helps manage prompt workflows and AI interactions, while the Groq API provides fast and intelligent response generation. Together, these technologies enable the system to perform advanced AI-based code analysis and documentation tasks effectively. This integration forms the foundation of the intelligent capabilities provided by the AI-Powered Code Reviewer and Quality Assistant system.

2.2.3 Validation and Metrics Module

The Validation and Metrics Module is developed to examine the overall quality and maintainability of the uploaded source code. Its main purpose is to help users identify documentation problems, evaluate software complexity, and understand the condition of the codebase more effectively. By automating these checks, the module minimizes manual review work and supports developers in following better coding and documentation practices during development.

One of the key tasks performed by this module is documentation validation. The system verifies whether functions, classes, and modules contain suitable docstrings and checks if the written documentation follows standard formatting conventions. It can detect missing, incomplete, or improperly structured docstrings and notify users about areas that require correction. This improves the consistency and readability of project documentation and helps maintain organized coding standards.

The module also carries out documentation coverage analysis. During this process, the application measures how much of the source code has been properly documented. It examines functions, classes, and methods individually and identifies sections where documentation is absent or insufficient. This allows developers to recognize undocumented parts of the project and improve overall documentation completeness.

Another important capability of the module is complexity analysis. The system calculates software metrics such as Cyclomatic Complexity to estimate how difficult a function or code segment may be to understand, test, or maintain. Code sections with higher complexity values are highlighted because they may become harder to debug or manage in the future. This helps developers simplify program logic and create cleaner implementations. The module additionally performs maintainability evaluation to determine how easily the software can be modified or extended later. Factors such as code structure, complexity level, and documentation quality are analyzed to generate maintainability-related insights. These results help developers understand the long-term quality of the project and encourage the development of more manageable software systems.

To support these operations, the system uses libraries such as pydocstyle and Radon. The pydocstyle library checks docstrings against standard documentation rules and identifies formatting-related issues, while Radon is used to calculate complexity measurements and maintainability indexes. By combining these tools, the module provides detailed analysis results that improve the reliability and effectiveness of software quality evaluation within the application.

2.2.4 Reporting and Storage Module

The Reporting and Storage Module handles the generation, visualization, and storage of all analysis results produced by the system. This module plays an important role in presenting software quality information in a structured and understandable format. It allows users to view detailed reports, track previous analysis records, and monitor improvements made to the source code over time. By organizing analysis data properly, the module helps developers and students manage project reviews more efficiently.

One of the main tasks performed by this module is generating reports in JSON format. After completing code analysis, the system gathers information related to documentation status, complexity measurements, maintainability results, validation output, and AI-generated explanations. This data is converted into structured JSON files that can be stored, reused, or processed later if needed. Using JSON format makes the reports lightweight, easy to read, and suitable for maintaining organized project records. The module also provides an analytics dashboard where users can view project insights through charts, graphs, and summary reports. The dashboard helps users understand the overall condition of the software by displaying metrics such as documentation coverage, complexity levels, and validation results visually. Interactive visualizations

make it easier to identify issues within the code and understand which parts of the project may require improvement or optimization. Another important feature of this module is maintaining review logs and analysis history. The system stores previous analysis records so users can compare results from different stages of development. This helps track improvements in code quality, documentation, and maintainability over time. Keeping analysis history is useful during software maintenance, project reviews, and academic evaluation, where monitoring project progress is important.

Overall, the Reporting and Storage Module improves the usability and effectiveness of the system by organizing generated data in a clear and manageable form. It supports better project monitoring and allows users to access analysis results whenever required, making the software review process more efficient and organized.

2.3 Data Flow and Module Communication

The Data Flow and Module Communication process explains how different modules of the AI-Powered Code Reviewer and Quality Assistant exchange information during code analysis and documentation generation. The system follows a sequential workflow where the output generated by one module becomes the input for another module. This organized communication between components ensures smooth execution of the entire analysis process and improves the efficiency of the application.

The process starts when the user uploads source code files through the Streamlit-based web interface. The uploaded files are first received by the Source Code Reader, which reads the contents of the files and forwards them to the Core Processing Module. Inside this module, AST parsing techniques are used to analyze the internal structure of the program. The parser extracts programming elements such as functions, classes, parameters, loops, return statements, conditional blocks, and exception handling structures. The extracted metadata is then prepared for further processing and shared with other modules in the system.

After the code structure is extracted, the information is sent to the AI Analysis and Documentation Module. This module creates structured prompts using the extracted metadata and communicates with the AI services integrated through Groq API and LangChain. The AI model processes the request and generates docstrings, explanations, and suggestions related to code readability and maintainability. The generated results are returned to the system and displayed to the user through the dashboard interface.

This interaction allows the application to provide intelligent and context-aware assistance for software documentation and code understanding.

At the same time, the extracted metadata is also transferred to the Validation and Metrics Module. This module checks documentation standards, calculates complexity metrics, measures maintainability, and performs coverage analysis using tools such as pydocstyle and Radon. The validation and metrics results are then forwarded to the Reporting and Visualization Module, where the information is converted into charts, summaries, and JSON reports. The dashboard displays these reports in an interactive format so users can easily understand the condition and quality of the source code.

The system also communicates with the Storage Module, where analysis reports, review logs, and previous records are saved for future use. This storage mechanism helps users compare project quality over different stages of development and maintain a history of software analysis activities. The modular communication structure of the system ensures efficient data transfer between all components and supports accurate processing, reporting, and long-term project monitoring.

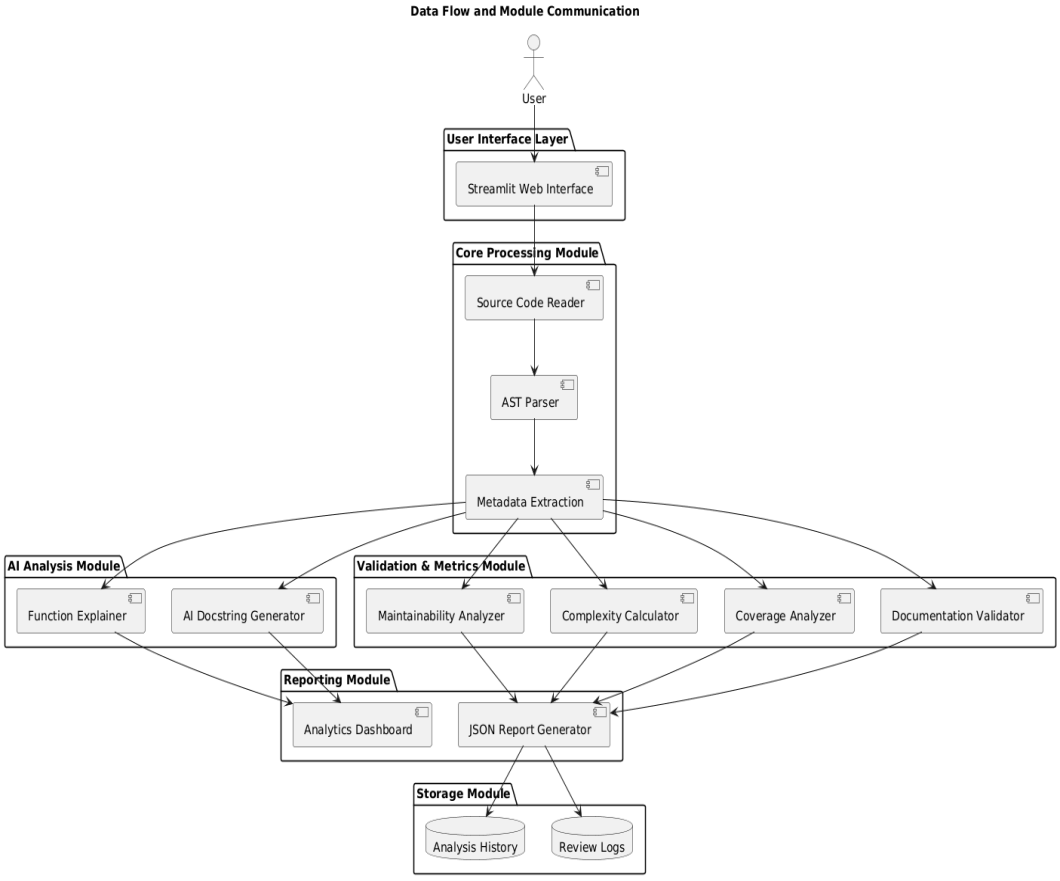


Fig.2.3 Data Flow and Module Communication

As shown in Figure 3.3, the system follows a structured workflow where both Python and Java source code files pass through multiple processing stages for analysis, documentation generation, validation, and report creation. The diagram shows how different modules communicate with each other to perform automated code review and software quality analysis efficiently. For Python programs, the system generates docstrings and performs code quality analysis, while for Java programs, the system supports automatic Javadoc generation and documentation assistance. The Core Processing Module extracts important programming structures such as functions, classes, methods, parameters, and control-flow information from both languages. This extracted information is then processed by the AI analysis and validation modules to generate meaningful documentation and quality reports. Finally, the generated outputs are displayed through the analytics dashboard and stored in the reporting system for future reference and project tracking.

2.4 Technology Stack Integration

The AI-Powered Code Reviewer and Quality Assistant is developed using multiple technologies and software tools that work together to provide intelligent code analysis, documentation generation, validation, and visualization features. The integration of these technologies helps the system perform efficiently while maintaining flexibility and scalability for future improvements. Each technology used in the project contributes to a specific part of the overall workflow and supports smooth communication between different modules of the application.

2.4.1 Python Backend

The backend of the AI-Powered Code Reviewer and Quality Assistant is developed using Python. It handles major system functionalities such as source code analysis, AST parsing, metadata extraction, AI integration, validation, metrics calculation, and report generation. Python was selected because of its simplicity, flexibility, and strong support for Artificial Intelligence and software analysis applications.

The backend processes both Python and Java source code files and manages communication between different modules of the system. It integrates external tools and libraries such as Groq API, LangChain, Radon, and pydocstyle to provide features like automated docstring generation, Java Javadoc assistance, documentation validation, and maintainability analysis.

Python also supports modular development, making the system easier to maintain, extend, and upgrade in the future. Its compatibility with visualization tools and AI services helps the application perform intelligent code analysis efficiently and reliably.

2.4.2 Streamlit Framework

The user interface of the AI-Powered Code Reviewer and Quality Assistant is developed using the Streamlit framework. Streamlit is a Python-based web framework used for creating interactive and user-friendly applications with minimal frontend development effort. It is suitable for building dashboards, analytical tools, and data-driven applications. In this project, Streamlit provides the interface through which users upload Python and Java source code files, generate documentation, view validation results, and access analytics dashboards. The framework supports interactive components such as buttons, navigation menus, charts, and file upload systems, which improve the overall user experience.

One of the main advantages of Streamlit is its direct integration with Python backend modules. This allows smooth communication between the frontend interface and backend processing system without requiring separate frontend technologies. Streamlit also supports real-time updates and dynamic visualization of reports, making the application responsive, lightweight, and easy to use.

2.4.3 Plotly Visualization

The AI-Powered Code Reviewer and Quality Assistant uses Plotly to generate interactive charts, graphs, and analytics dashboards for software analysis results. Plotly helps present complex information such as code complexity, maintainability index, documentation coverage, and validation summaries in a visual and easy-to-understand format. Instead of displaying only text-based reports, the system converts analysis data into graphical representations that help users quickly understand the overall quality of the source code.

Plotly is integrated with the Streamlit dashboard to provide dynamic and interactive visualizations for both Python and Java projects. Users can view updated analysis reports, interact with charts, and monitor software quality metrics directly through the web interface. The visualization features improve readability of reports and make the application more user-friendly by simplifying software analysis and helping developers identify areas that may require improvement or optimization.

2.5 External Libraries and APIs

The AI-Powered Code Reviewer and Quality Assistant integrates several external libraries and APIs to support intelligent code analysis, automated documentation generation, validation, and software quality evaluation. These technologies help the system process both Python and Java source code efficiently while reducing manual effort during software development and maintenance.

2.5.1 Groq API

The Groq API is integrated into the system to provide Artificial Intelligence capabilities for documentation generation and code explanation. It is used to generate Python docstrings, Java Javadocs, and human-readable explanations for functions and methods. The API processes structured prompts prepared by the system and returns meaningful documentation and suggestions based on the uploaded source code.

2.5.2 LangChain Integration

LangChain is used to manage communication between the application and the AI model. It helps organize prompts, handle AI responses, and maintain smooth interaction between backend processing modules and the language model. This integration improves the quality and consistency of generated documentation and explanations.

2.5.3 AST Module

The AST (Abstract Syntax Tree) module is used for analyzing Python source code structure. It converts source code into a tree-like representation that helps the system identify functions, classes, parameters, loops, conditions, and return statements. This structural analysis forms the foundation for metadata extraction, validation, and AI-powered documentation generation.

2.5.4 Pydocstyle Library

The pydocstyle library is integrated for validating Python documentation standards. It checks whether functions, classes, and modules contain proper docstrings and whether the documentation follows standard formatting rules. This library helps improve documentation consistency and maintain proper coding standards within Python projects.

2.5.5 Java Standard Library Support

For Java source code processing, the system currently uses Java Standard Library packages such as java.util.*. The analyzed Java files mainly use built-in collection and utility classes including List, ArrayList, Set, HashSet, Map, HashMap, and Arrays.

These standard libraries are used for handling data structures, collections, and utility operations within Java programs. The system processes Java methods and classes to generate Javadoc documentation and provide AI-assisted explanation features without requiring external Java frameworks or third-party libraries.

2.5.6 Radon Library

The Radon library is used to calculate software quality metrics such as cyclomatic complexity and maintainability index. These metrics help identify complex or difficult-to-maintain code sections and provide insight into the overall quality of the software project.

Chapter 3

FEATURES & FUNCTIONALITY

The AI-Powered Code Reviewer and Quality Assistant includes several features that help improve software documentation, readability, and overall code quality. The application supports both Python and Java source code and uses automated analysis along with AI-assisted processing to simplify different software review tasks. By automating repetitive activities such as documentation generation, validation, and code analysis, the system helps reduce the amount of manual work required during development.

One of the major functionalities of the project is automatic generation of Python docstrings and Java Javadocs. After analyzing functions and methods from the uploaded source code, the application creates readable documentation using AI-based processing techniques. This helps maintain better documentation standards and makes the source code easier to understand for developers and project teams. The project also provides support for syntax error identification and correction assistance. During analysis, the application detects syntax-related issues present in the code and provides suggestions that help users correct programming mistakes more efficiently. This feature improves code reliability and helps developers identify errors at an earlier stage.

Another important capability of the system is code optimization support. The application examines program structure and logic to identify sections that can be simplified or improved. Based on the analysis, it provides optimization suggestions that help reduce unnecessary complexity and improve maintainability of the source code. These recommendations assist developers in writing cleaner and more organized programs.

Along with documentation and optimization features, the system also performs validation and software quality analysis. It checks coding standards, documentation coverage, maintainability, and complexity-related metrics using validation and analysis libraries integrated within the application. The generated reports and visual dashboards help users understand the condition of the project more clearly and identify parts of the code that may require further improvement or restructuring.

3.1 Code Analysis

The AI-Powered Code Reviewer and Quality Assistant analyzes source code to understand the structure and logic of programs before generating documentation,

validation reports, and optimization suggestions. The system supports both Python and Java source code and uses different parsing techniques for each language. This analysis process helps the application extract important programming information that is later used for AI-generated documentation, quality analysis, and code understanding.

3.1.1 Code Parsing for Python using AST

For Python files, the system uses Python's built-in Abstract Syntax Tree (AST) module. The AST process converts source code into a structured tree format that represents the logical arrangement of the program. This allows the application to identify functions, classes, loops, conditions, return statements, decorators, and exception handling blocks more accurately. Instead of reading code as simple text, the system understands the structure of the program, which improves the quality of analysis and generated docstrings.

3.1.2 Code Parsing for Java using Regex

For processing Java source code, the application uses Regex-based analysis instead of AST parsing. Since Python does not include a built-in parser for Java programs, regular expressions are used to recognize important programming structures such as classes, methods, constructors, parameters, and access modifiers. The system also monitors opening and closing braces to correctly determine the boundaries of classes and methods within Java files. This method offers a simpler and lightweight solution for Java code processing without relying on large external parsing frameworks.

During analysis, the module separates functions, methods, and classes from the uploaded source code so that each component can be examined independently. The system additionally gathers important metadata including method names, parameters, return types, exception details, and existing documentation information. This structured data is later used by validation and AI-related modules for further processing.

The module also studies the control flow of the program by examining loops, conditional statements, return blocks, and exception-handling structures. This helps the application understand how the program behaves during execution and assists in evaluating code complexity and maintainability. The collected information supports accurate documentation generation, validation checks, and software quality analysis for both Python and Java projects

3.2 AI Docstring and Javadoc Generation

The AI-Powered Code Reviewer and Quality Assistant provides intelligent documentation generation features for both Python and Java source code using Large Language Models (LLMs). The system automatically generates Python docstrings and Java Javadocs by analyzing functions, methods, parameters, return statements, exceptions, and overall program structure. This feature reduces the manual effort required for writing technical documentation and helps developers maintain consistent documentation standards throughout the project.

3.2.1 LLM Integration (Groq via LangChain)

The system integrates Groq API through LangChain to provide context-aware documentation generation. After analyzing the uploaded source code, the application prepares structured prompts containing metadata such as function names, parameters, return values, exceptions, and control-flow information. These prompts are sent to the AI model, which generates meaningful and human-readable docstrings or Javadocs based on the functionality of the code. LangChain helps manage prompt handling and communication between the backend modules and the AI service efficiently.

3.2.2 Style Support (Google, NumPy, reST)

The system provides support for different documentation formats so users can generate docstrings according to their preferred coding standards. Depending on project requirements, the generated documentation can be created in formats such as Google Style, NumPy Style, or reStructuredText (reST). This flexibility allows the application to adapt to different development environments and documentation practices.

While generating documentation, the application can automatically create important sections including Arguments, Returns, Raises, Yields, and Examples by using the information extracted from the source code. Organizing the documentation in this structured manner improves readability and helps maintain clear and consistent documentation for both Python and Java programs.

3.2.3 Auto-Apply and Batch Processing Feature

The system supports automatic application of generated documentation directly into source code files. Users can review the generated docstrings or Javadocs through the live preview interface before applying them to the codebase. The application also supports batch processing, allowing documentation generation for multiple functions

or methods simultaneously. This feature improves productivity and reduces the time required for documenting large projects manually.

3.3 Documentation Validation

The AI-Powered Code Reviewer and Quality Assistant includes a documentation validation system that helps users maintain clear, organized, and properly formatted documentation within their projects. The module checks existing docstrings and Javadocs to identify missing or incomplete documentation and ensures that the written documentation follows standard formatting rules. This feature helps improve readability of the source code and makes software projects easier to understand and maintain.

3.3.1 Coverage Analysis

The system performs coverage analysis to measure how much of the source code is properly documented. It checks functions, classes, and methods individually and calculates documentation coverage for each file as well as for the entire project. The application automatically detects functions or classes that do not contain docstrings or Javadocs and displays them in the analysis results. An interactive coverage table is also provided to help users view the documentation status of different parts of the project more easily.

3.3.2 Style Compliance Checking

The system includes a style validation feature that checks whether the generated or existing documentation follows standard documentation formats such as Google Style, NumPy Style, and reStructuredText format. It examines important documentation sections like parameters, return values, exceptions, and examples to ensure that the structure remains clear and consistent throughout the project. This helps maintain better readability and organized documentation for both Python and Java source code.

To improve documentation quality, the application also validates docstrings using PEP 257 documentation rules through pydocstyle integration. During analysis, the system identifies missing or incorrectly formatted docstrings and displays related validation codes directly in the interface. Some commonly detected validation rules include:

- D100 — Missing docstring in public module
- D101 — Missing docstring in public class
- D102 — Missing docstring in public method
- D103 — Missing docstring in public function

- D200 — One-line docstring should fit on one line
- D205 — Blank line required between summary and description
- D400 — First line should end with a period
- D401 — First line should be written in imperative form

These validation checks help developers quickly identify documentation issues and maintain proper documentation standards across the project. By automatically detecting formatting problems and missing documentation, the system reduces manual checking effort and improves the overall quality and consistency of the codebase.

3.3.3 pydocstyle Integration

The system uses the pydocstyle library to validate Python docstrings automatically. The library identifies formatting problems, missing documentation, and style-related issues based on standard documentation guidelines. The detected issues are displayed directly in the application interface so users can review and correct them easily. This automated validation process reduces manual checking effort and helps maintain better documentation quality across the project.

3.4 Auto-Fix and Code Update Feature

The AI-Powered Code Reviewer and Quality Assistant provides an Auto-Fix feature that helps users apply generated documentation and code updates directly into the source files automatically. This feature reduces the need for manual editing and makes the process of updating large codebases faster and more convenient. The system is designed to insert changes carefully without affecting the original organization of the code.One of the important capabilities of this module is the One-Click Apply feature. After generating docstrings or Javadocs, the system can place them automatically at the correct locations inside Python or Java files. Users do not need to manually copy and paste the generated documentation, which saves time and reduces the chance of editing mistakes.

The module also supports batch processing, allowing documentation fixes to be applied to multiple files in a single operation. This is useful for projects containing many undocumented functions or methods because it helps developers update several files quickly instead of handling each file separately.

While updating files, the system maintains the original formatting, indentation, spacing, and overall structure of the code. This helps preserve code readability and prevents unnecessary formatting changes after modifications are applied. The update

process is handled carefully so the existing program structure remains stable and organized.

The Auto-Fix feature also keeps records of applied changes, allowing modifications to be reviewed or reversed later if required. This provides safer file handling and gives users more control over automatic updates made by the system.

3.5 AI-Powered Assistance Features

The AI-Powered Code Reviewer and Quality Assistant includes several AI-based features that simplify source code analysis and maintenance activities. These functionalities help users understand program behavior, improve code organization, generate documentation, validate source code, and detect possible issues with reduced manual effort. The application supports both Python and Java programs and provides intelligent assistance by studying the structure and execution flow of the uploaded code.

3.5.1 Function Explainer

The Function Explainer feature helps users understand the purpose and behavior of functions or methods present in the source code. The system studies the function logic, parameters, return values, and execution flow to generate explanations in simple and understandable language. This feature is helpful when working with large or unfamiliar codebases because it allows users to understand program functionality without manually analyzing every line of code. It also supports students and beginners in learning programming concepts more effectively.

3.5.2 AI Optimizer

The AI Optimizer is designed to improve the readability and maintainability of source code by suggesting better coding approaches and structural improvements. The system examines the uploaded code to identify repetitive logic, unnecessary complexity, and inefficient coding patterns. Based on the analysis, it provides optimized suggestions while preserving the original functionality of the program. This helps developers write cleaner and more organized code.

3.5.3 Bug Fix Suggester

The Bug Fix Suggester helps users detect possible errors and issues within the source code. The system analyzes the code structure and identifies syntax mistakes, logical problems, and other common programming issues that may affect software performance or reliability. It also provides recommendations and corrected solutions to help users resolve problems more efficiently and reduce debugging time.

3.5.4 Python AI Assistant

The Python AI Assistant is designed to provide interactive help for Python-related programming activities. Through this feature, users can understand how a program works, ask coding-related questions, and receive AI-based suggestions for improving or debugging Python code. The assistant supports developers and students by offering explanations, guidance, and recommendations based on the uploaded source code, making coding and learning tasks easier to manage.

3.5.5 Java Assistant

The Java Assistant is designed to provide support for analyzing Java source code and generating documentation automatically. It examines Java classes, constructors, and methods to create readable explanations and produce Javadoc comments based on the structure of the program. This functionality helps improve the clarity of documentation and makes Java applications easier to understand, manage, and maintain, particularly when working with large codebases.

3.5.6 Java Validator

The Java Validator is designed to examine Java source code and review both its structure and documentation quality. It detects issues such as missing Javadoc comments, incomplete method details, and inconsistencies present in classes or methods. By identifying these problems during analysis, the feature helps maintain cleaner and more organized Java code, making the application easier to read, understand, and manage.

3.6 Technical Capabilities

The AI-Powered Code Reviewer and Quality Assistant is designed with multiple technical features that help the system perform code analysis and documentation tasks efficiently. These capabilities allow the application to work with different programming languages, process multiple source files, manage project data smoothly, and interact with AI services securely. The system is built to provide stable performance, better usability, and flexibility while handling software analysis, validation, optimization, and documentation generation for both Python and Java projects.

3.6.1 Multi-Language Support

The application is developed to support both Python and Java source code within a single platform. Users can generate Python docstrings, create Java Javadocs, validate code structure, and perform software quality analysis without switching between different tools. This makes the system more flexible and useful for projects that involve multiple programming languages.

3.6.2 Batch File Processing

The system allows multiple source code files to be analyzed together in one process. Users can upload several files or complete project folders instead of processing files individually. This feature saves time and improves efficiency, especially when working on large projects containing many functions, methods, and classes.

3.6.3 Automatic Encoding Detection

The application can automatically identify different file encoding formats such as UTF-8, UTF-16, Latin-1, and CP1252 while reading source code files. This helps prevent encoding-related issues and allows the system to process files created from different editors and development environments smoothly.

3.6.4 Workspace Navigation

The system provides workspace navigation support that allows users to browse and analyze entire project directories. Instead of uploading files one by one, users can work directly with complete codebases. This makes project-level analysis simpler and more convenient for larger software projects.

3.6.5 Environment Integration

The project uses environment-based configuration to handle external service settings and API-related information in a secure manner. Sensitive details such as GROQ API keys are stored in .env files instead of being written directly into the main program files. This approach helps keep confidential information organized, improves security, and separates configuration settings from the application source code.

3.6.6 Error Handling

The project includes error-handling mechanisms to manage issues that may occur during file processing, analysis, or AI operations. Instead of stopping execution completely, the system provides clear and user-friendly error messages that help users understand and resolve problems more easily. This improves the reliability and stability of the application during usage.

Chapter 4

SYSTEM WORKFLOW AND PROCESSING

The AI-Powered Code Reviewer and Quality Assistant follows a structured processing workflow to perform source code analysis, documentation generation, validation, optimization, and report creation. The system is designed so that each module performs a specific operation while maintaining smooth communication with other components. This organized workflow helps the application process Python and Java source code efficiently and generate accurate analysis results.

The workflow begins when users upload source code files through the Streamlit-based interface. After receiving the files, the system reads the source code, detects the file encoding format, and prepares the content for further processing. The uploaded code is then analyzed to identify programming structures such as functions, classes, methods, parameters, loops, conditions, and exception handling blocks. The extracted information is converted into structured metadata that can be used by different processing modules within the application.

Once the source code structure is prepared, the metadata is forwarded to the AI processing modules. These modules communicate with the Groq API through LangChain to generate Python docstrings, Java Javadocs, function explanations, optimization suggestions, and validation-related outputs. At the same time, the validation and metrics modules examine the source code to calculate complexity, maintainability, documentation coverage, and coding standard compliance.

After processing is completed, the system generates reports, visual summaries, and interactive dashboards using Streamlit and Plotly. The analysis results are displayed in a clear and organized format so users can easily understand the condition of the source code. The generated reports and analysis records are also stored in JSON format for future reference, comparison, and project tracking purposes.

4.1 Input File Handling Process

The system begins processing when users upload Python or Java source code files through the web interface. It is capable of handling single files, multiple files, or complete project folders, which makes the analysis process more convenient for both small and large projects. This approach helps users work with entire codebases without repeatedly uploading files one by one.

After the files are received, the application reads the source code and checks the encoding format before starting further operations. Different encoding types such as UTF-8, UTF-16, Latin-1, and CP1252 are supported to ensure that files created from different editors and environments can be processed correctly without generating reading errors.

Once the files are verified, the system organizes them according to their programming language and prepares them for analysis. Python files are forwarded to the AST-based processing module, while Java files are sent to the Regex-based parser. This structured handling process helps the application perform accurate analysis, documentation generation, and validation operations efficiently for both Python and Java projects.

4.2 Source Code Parsing Sequence

After the uploaded files are received and prepared, the application begins examining the internal structure of the source code through a parsing process. During this stage, the system identifies important programming elements such as functions, classes, methods, loops, conditions, and exception-handling blocks. Instead of treating the code as plain text, the parser studies the logical organization of the program, which helps improve analysis accuracy.

For Python source code, the application uses Python's built-in AST (Abstract Syntax Tree) module. This process converts the program into a tree-like structure that represents relationships between different programming components. Using AST-based analysis allows the system to understand code structure more effectively and supports accurate documentation generation, validation, and source code interpretation.

For Java source code, the project applies Regex-based parsing techniques to identify classes, constructors, methods, parameters, and access modifiers. The system also monitors opening and closing braces to correctly determine the boundaries of classes and methods within Java files. This provides a lightweight solution for Java code processing without depending on additional external parsers or frameworks.

After parsing is completed, the extracted information is arranged into structured data formats and transferred to other modules within the application. The processed data is then used for operations such as metadata extraction, AI-generated documentation, validation checks, complexity analysis, and report generation.

4.3 Internal Data Processing

After the parsing stage is completed, the system processes the extracted information and prepares it for further analysis. During this step, important details such as function names, class names, parameters, return values, exceptions, loops, and conditional statements are collected and arranged in a structured format. This organized processing helps different modules of the application work with the source code information more efficiently.

The processed data is shared between multiple system modules responsible for documentation generation, validation, metrics calculation, optimization, and report preparation. Each module receives only the required information needed for its operation, which helps maintain smooth communication and improves the overall efficiency of the system. For Python source code, the information extracted through AST parsing is converted into structured metadata objects. For Java source code, the data collected through Regex-based analysis is also organized into a similar structured format. This processing helps the system generate better docstrings, Javadocs, explanations, and software quality reports.

Once the internal processing is completed, the prepared metadata is forwarded to the AI analysis and validation modules for further operations. This stage acts as an important link between source code analysis and intelligent processing within the application.

4.4 AI Request and Response Cycle

After the source code information is processed, the system prepares AI requests using the extracted metadata from the uploaded files. Details such as function names, parameters, return values, exceptions, and control-flow structures are arranged into structured prompts so the AI model can understand the logic and purpose of the code more effectively.

The application connects with the Groq AI service through the LangChain framework to perform different AI-based operations. These operations include generating Python docstrings, creating Java Javadocs, explaining functions, suggesting optimizations, and assisting with bug detection. LangChain helps manage the communication process between the backend modules and the AI model while handling prompt formatting and response management efficiently.

Once the request is sent, the AI model analyzes the provided information and generates responses based on the selected operation. The returned output may include documentation, simplified code explanations, optimization suggestions, validation support, or corrected code recommendations. The generated responses are designed to be readable and easier for users to understand.

After receiving the AI response, the system processes and formats the generated content before displaying it on the dashboard interface. Users can review the generated documentation, explanations, and suggestions before applying changes to the source code. This complete request and response cycle allows the application to provide intelligent and context-aware support for both Python and Java development tasks.

4.5 Analysis Result Processing

Once the analysis and AI-related operations are completed, the system processes the generated results and prepares them for display. Information collected from different modules such as documentation generation, validation, metrics analysis, optimization, and bug detection is organized into a clear and structured format. This helps the application present the results in a way that is easier for users to understand and review. The system separates outputs based on their type, including generated docstrings, Javadocs, validation messages, complexity values, maintainability scores, and AI-generated suggestions. Each result is formatted properly before being displayed through the dashboard interface. This organized arrangement improves readability and allows users to examine different analysis results more efficiently.

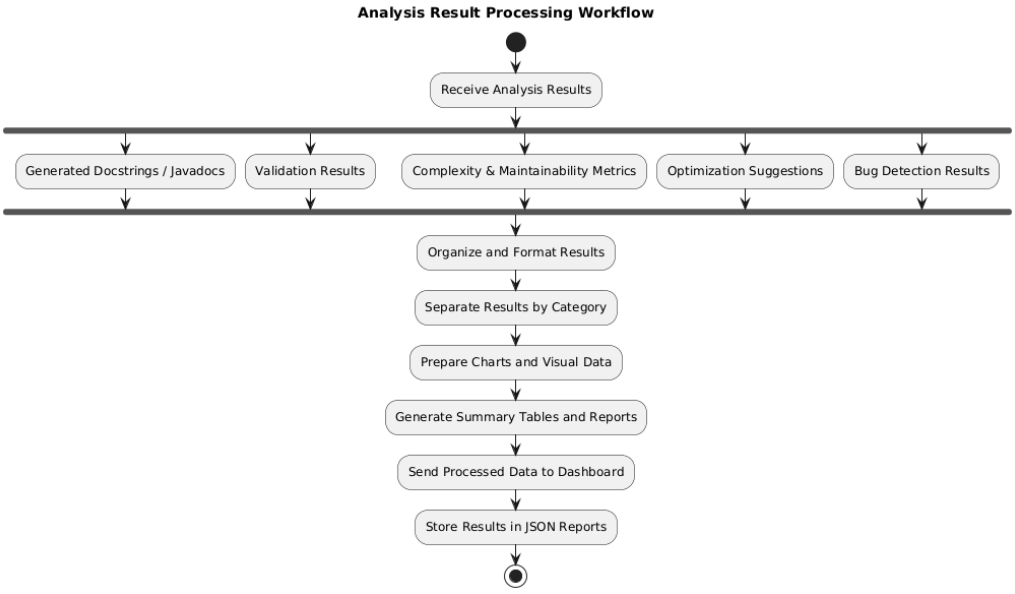


Fig.4.5 Analysis Result Processing Workflow

During this stage, the application also prepares data for visual representation. Metrics such as documentation coverage, complexity analysis, and maintainability scores are converted into charts, graphs, and summary tables to simplify software quality analysis. These visual representations help users quickly identify important insights and areas that may require improvement within the source code.

After processing and formatting are completed, the final results are forwarded to the reporting and visualization modules. The generated outputs are displayed on the interactive dashboard and stored for future reference, comparison, and project tracking.

4.6 Report Compilation Process

After all analysis operations are completed, the system compiles the generated outputs into organized reports for the user. Information collected from documentation generation, validation checks, complexity analysis, maintainability evaluation, optimization suggestions, and bug detection modules is combined into a structured report format. This process helps present software analysis results in a clear and understandable manner.

The system arranges the collected data into different sections so users can review project quality more efficiently. Documentation coverage details, validation messages, complexity metrics, AI-generated explanations, and optimization suggestions are grouped properly before final report generation. This organized structure improves readability and allows users to analyze project information more effectively.

During report compilation, the application also prepares visual summaries such as charts, graphs, and tables using the processed analysis data. These visual elements help users quickly understand the overall condition of the source code and identify areas that may require improvement. Both file-level and project-level analysis information can be included within the generated reports.

Once the compilation process is completed, the reports are displayed through the dashboard interface and stored in JSON format for future reference. The stored reports allow users to track project improvements, compare analysis results over time, and maintain organized records of software quality evaluation activities.

4.7 Data Storage and Retrieval

The system stores generated analysis results and reports so users can access them later whenever required. Information such as generated docstrings, Javadocs, validation

results, complexity measurements, maintainability scores, optimization suggestions, and bug analysis outputs is saved after the processing stages are completed. This helps maintain proper records of software analysis activities within the application.

The application mainly uses JSON format for storing reports and review data. JSON provides a simple and organized way to manage analysis information while keeping the stored data lightweight and easy to read. This approach helps the system handle project records efficiently and simplifies the process of saving and retrieving analysis history.

Users can retrieve previously generated reports through the application to review earlier analysis results and compare project improvements over time. Stored records help track documentation updates, code quality changes, and optimization progress during different stages of software development. This feature is useful for project monitoring, maintenance activities, and academic evaluation purposes.

By maintaining organized storage and retrieval operations, the system supports better management of analysis data and helps users monitor the overall progress and quality of their software projects more effectively.

4.8 Execution Sequence of the System

The system follows a step-by-step execution process where each module performs a specific operation before passing the processed information to the next stage. This organized sequence helps the application handle source code analysis, documentation generation, validation, and reporting smoothly while maintaining proper communication between all system components.

The execution begins when users upload Python or Java source code files through the web interface. After receiving the files, the application reads the source code, detects the encoding format, and prepares the files for analysis. The uploaded files are then separated according to their programming language so they can be processed by the appropriate analysis modules.

During the next stage, Python files are analyzed using AST-based parsing, while Java files are processed using Regex-based analysis methods. The system extracts important programming information such as functions, classes, methods, parameters, loops, conditions, and exception handling structures. This extracted information is organized into structured metadata for further processing.

Once the metadata is prepared, it is forwarded to the AI processing modules where the system generates docstrings, Javadocs, code explanations, optimization suggestions, and validation-related outputs. At the same time, the validation and metrics modules calculate complexity values, maintainability scores, and documentation coverage to evaluate the quality of the source code.

After all processing stages are completed, the generated results are formatted and displayed through the dashboard interface using charts, graphs, and summary reports. The final analysis records are also stored in JSON format so users can access previous reports and compare project improvements over time. This complete execution sequence allows the system to perform intelligent software analysis and documentation tasks efficiently for both Python and Java projects.

4.9 Overall Processing Pipeline

The AI-Powered Code Reviewer and Quality Assistant follows a connected processing pipeline where different modules work together to analyze source code, generate documentation, validate software quality, and prepare reports. The pipeline is organized in a sequential manner so that information produced at one stage can be used efficiently by the next stage. This structured workflow helps the system process both Python and Java source code smoothly and accurately.

The pipeline begins when users upload source code files through the application interface. After the files are received, the system reads the source code, checks the file encoding, and prepares the content for further processing. The files are then separated based on their programming language and sent to the corresponding parsing modules for analysis.

During parsing, the application identifies important programming structures such as functions, classes, methods, parameters, loops, conditions, and exception handling blocks. The extracted information is organized into structured metadata, which becomes the foundation for later operations such as AI-based documentation generation, validation, complexity analysis, and optimization.

The processed metadata is then forwarded to the AI modules connected through the Groq API and LangChain framework. These modules generate Python docstrings, Java Javadocs, function explanations, optimization suggestions, and other AI-assisted outputs. At the same time, validation and metrics modules evaluate documentation

quality, maintainability, and code complexity to measure the overall condition of the software project.

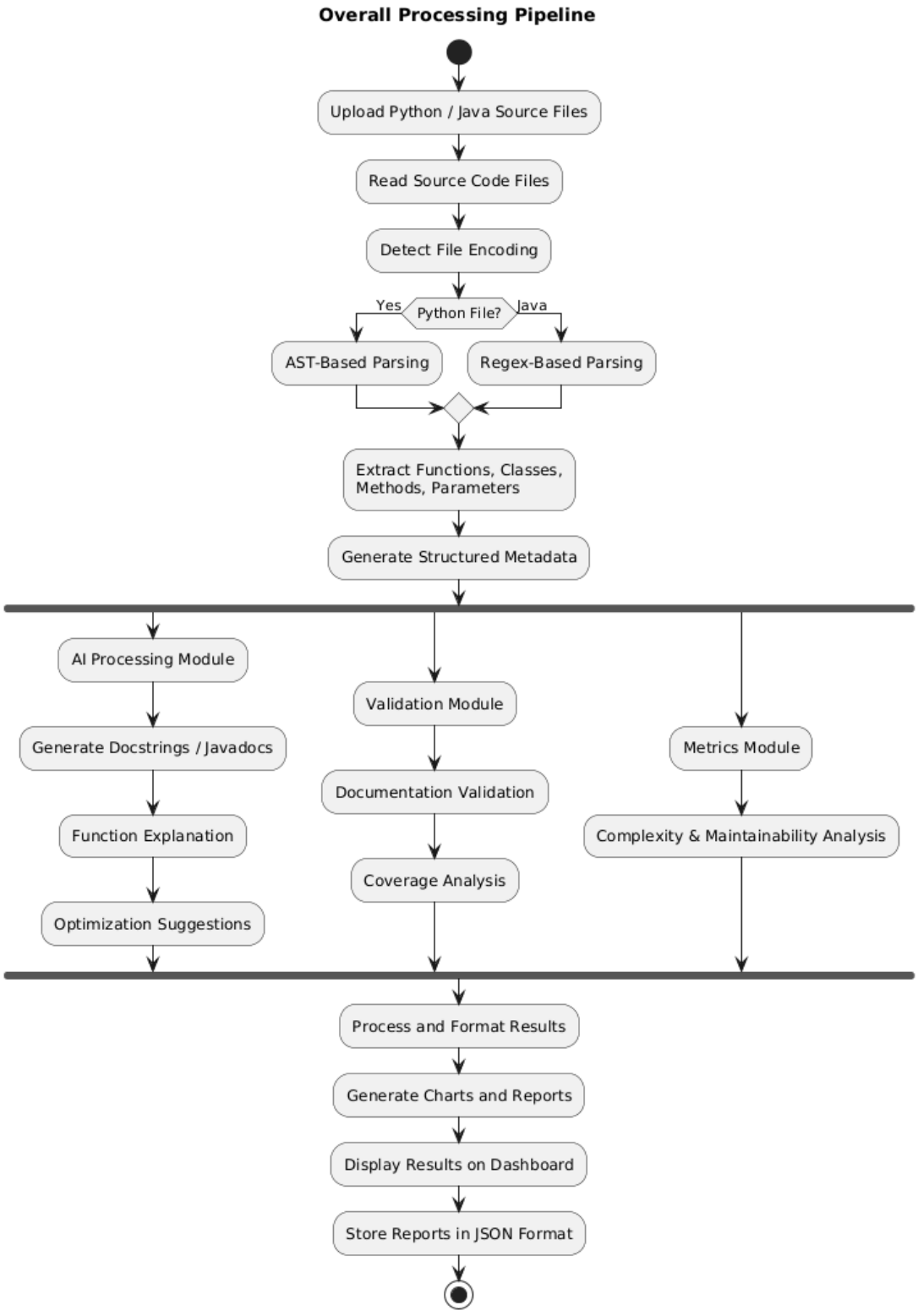


Fig.4.9 Analysis Result Workflow

After all analysis stages are completed, the generated outputs are formatted and displayed through interactive dashboards using charts, graphs, and summary reports. The final reports and analysis records are also stored in JSON format so users can review previous analysis results and track project improvements over time. This complete processing pipeline allows the system to provide organized, efficient, and intelligent software analysis support for both Python and Java projects.

Chapter 5

USER INTERFACE AND IMPLEMENTATION

The AI-Powered Code Reviewer and Quality Assistant is designed with a clean and interactive user interface that allows users to perform code analysis and documentation tasks easily. The application interface is developed using the Streamlit framework, which provides a lightweight and responsive environment for interacting with different features of the system. The overall design focuses on simplicity so users can work with the application comfortably without needing complex technical knowledge.

The interface is divided into multiple sections where users can upload Python or Java source code files, access analysis tools, generate documentation, validate code quality, and review reports. Navigation menus, buttons, upload components, and result panels are arranged in an organized manner to make the workflow easier to follow. This structured layout helps users move between different operations smoothly while working on software analysis tasks.

To improve readability of analysis results, the application displays information using charts, graphs, summary tables, and interactive dashboards created with Plotly. These visual elements help users understand complexity metrics, documentation coverage, maintainability analysis, and validation results more clearly. The dashboard presentation simplifies software quality evaluation and makes the generated outputs easier to interpret.

The interface also supports smooth communication between the frontend and backend modules of the system. Features such as real-time updates, session management, organized result displays, and clear error messages improve the overall user experience while maintaining stable application performance. This user-friendly design helps developers, students, and software teams work more efficiently with both Python and Java projects.

5.1 Home Page

The Home page acts as the starting point of the AI-Powered Code Reviewer and Quality Assistant. It provides users with a simple and organized interface for uploading Python and Java source code files and accessing different features of the system. The page is designed to make navigation easier and help users begin code analysis operations quickly.

The home interface includes file upload options that allow users to upload single files, multiple files, or complete project folders for analysis. It also provides quick access shortcuts to important modules such as documentation generation, validation, AI assistance, optimization, and analytics dashboards. These shortcuts help users move between different operations efficiently without navigating through multiple pages.

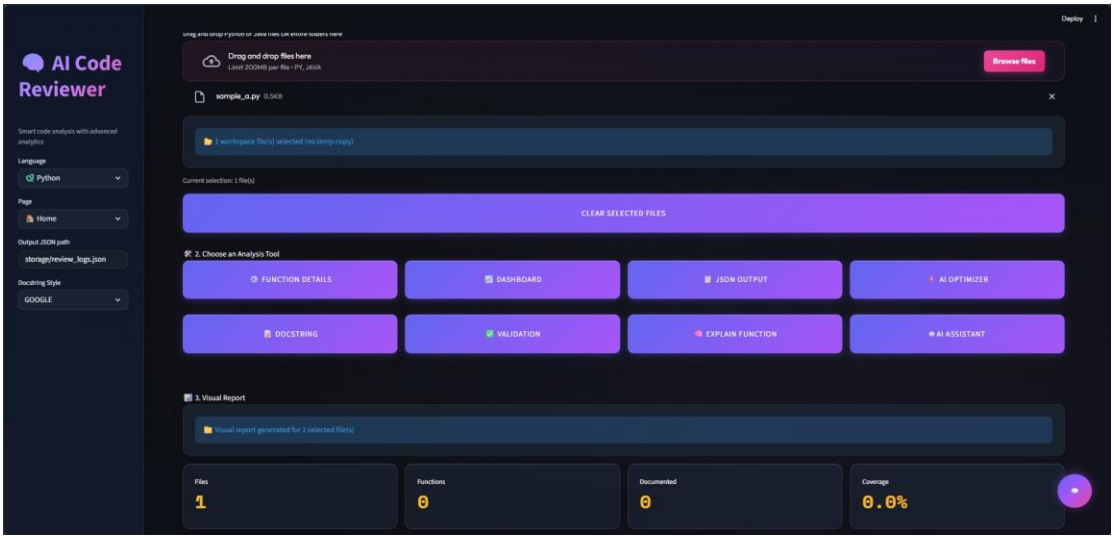


Fig 5.1: Home Page

In addition to navigation features, the Home page also displays basic project information and analysis summaries. Users can view uploaded file details, processing status, and recent analysis activity directly from the main dashboard. This organized layout improves usability and provides a smooth starting point for working with software analysis and documentation tasks within the application.

5.2 Docstring Generation Interface

The Docstring Generation interface helps users create documentation for Python functions and Java methods automatically using AI-based processing. The system studies the selected source code and generates meaningful docstrings or Javadocs based on function behaviour, parameters, return values, and overall program logic. This reduces the time and effort required for writing documentation manually.

The interface allows users to generate documentation for individual functions or methods separately. Generated docstrings and Javadocs are displayed in a preview section so users can review the content before applying it to the source code. This preview feature gives users better control over the generated documentation and allows corrections or modifications if needed.

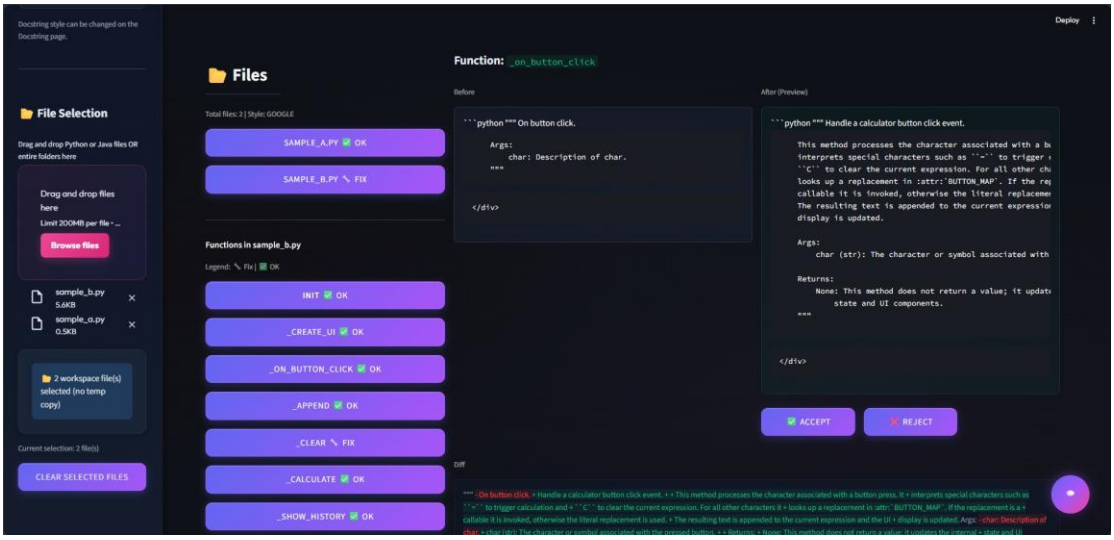


Fig 5.2: Docstring Generation Page

The page also provides options to apply the generated documentation directly into the source files. While updating the code, the system maintains proper indentation, formatting, and code structure to avoid unnecessary changes. This makes the documentation workflow smoother and more convenient for both Python and Java development projects.

5.3 Function Details Interface

The Function Details interface allows users to view detailed information about individual Python functions and Java methods identified during source code analysis. This section helps users understand the structure, behavior, and purpose of specific parts of the program without manually examining the entire source file. It is helpful for code understanding, debugging, documentation review, and maintenance activities.

For Python programs, the system displays information extracted through AST-based analysis, including function names, parameters, decorators, return statements, exceptions, loops, conditions, and available docstrings. This helps users understand the internal logic and flow of Python functions more clearly.

For Java programs, the interface shows details extracted using Regex-based analysis methods. Information such as class names, method names, constructors, parameters, return types, access modifiers, and Javadocs is presented in an organized format. The system also identifies method and class boundaries to improve understanding of Java source code structure.

All extracted details are displayed in a clean and structured manner so users can easily examine individual functions or methods. This organized presentation improves

code readability and helps developers, students, and software teams work more efficiently with both Python and Java projects.

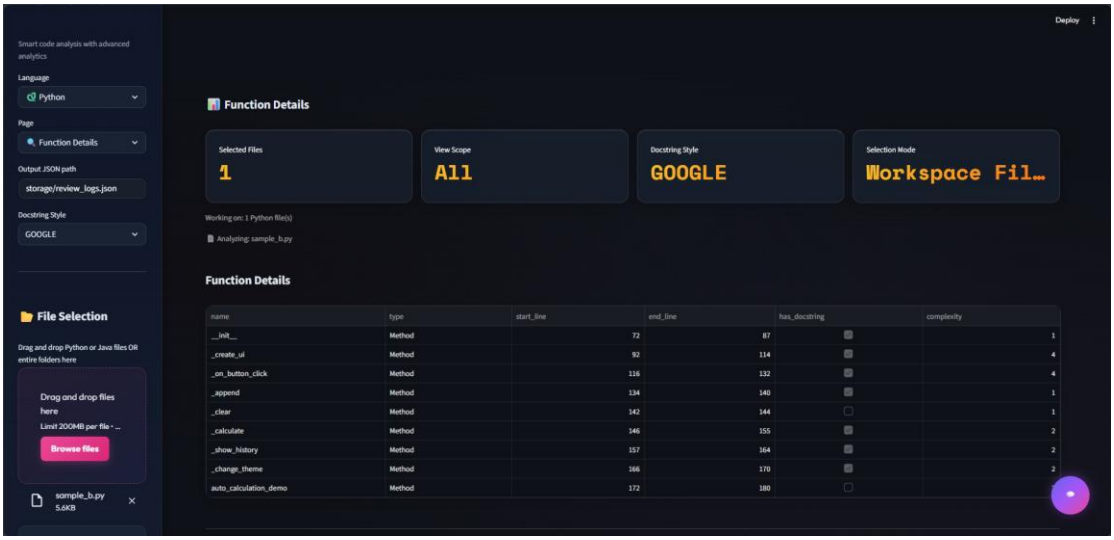


Fig 5.3 Function Details Page

5.4 Explain Function Interface

The Explain Function interface helps users understand the working and purpose of Python functions and Java methods through AI-generated explanations. Instead of manually studying every line of code, users can select a function or method and receive a simple explanation describing what the code does, how it works, and what kind of output it produces. This makes understanding complex or unfamiliar code much easier.

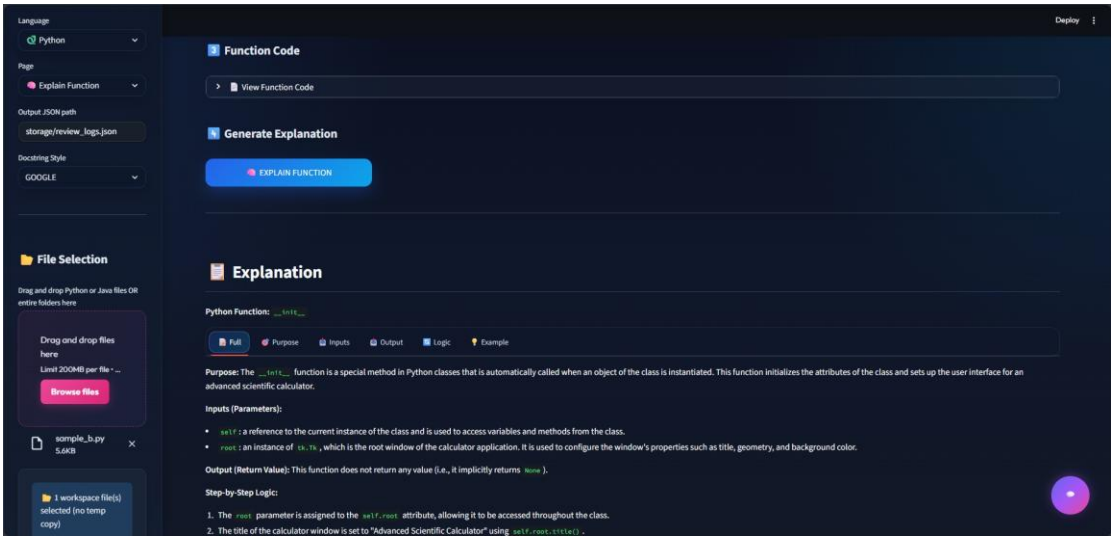


Fig 5.4: Function Explanation page

For Python programs, the system uses AST-based analysis to study function structure, parameters, loops, conditions, return statements, and exception handling

blocks. Using this information, the AI module generates explanations that describe the logic and behavior of the selected Python function in a more understandable way.

For Java programs, the application analyzes classes, methods, constructors, parameters, return types, and access modifiers using Regex-based processing techniques. The extracted details are then used to generate explanations that help users understand the functionality and structure of Java methods and classes more clearly.

The generated explanations are shown in a clean and organized format within the interface. This feature is especially useful for students, developers, and software teams who need to understand program logic quickly while working with large or unfamiliar Python and Java projects.

5.5 Validation Interface

The Validation interface helps users examine the quality and documentation status of both Python and Java source code. The system checks whether the uploaded files follow proper documentation standards, coding structure, and formatting rules. It also helps users identify missing documentation, style-related issues, and syntax errors that may affect the readability and maintainability of the project.

For Python source code, the interface displays docstring coverage details and validation results using the pydocstyle library. The system identifies missing or incorrectly written docstrings and shows validation rule codes such as D100, D101, D102, and D401. Coverage tables are also provided so users can easily see which functions or classes are documented properly and which sections still require documentation updates.

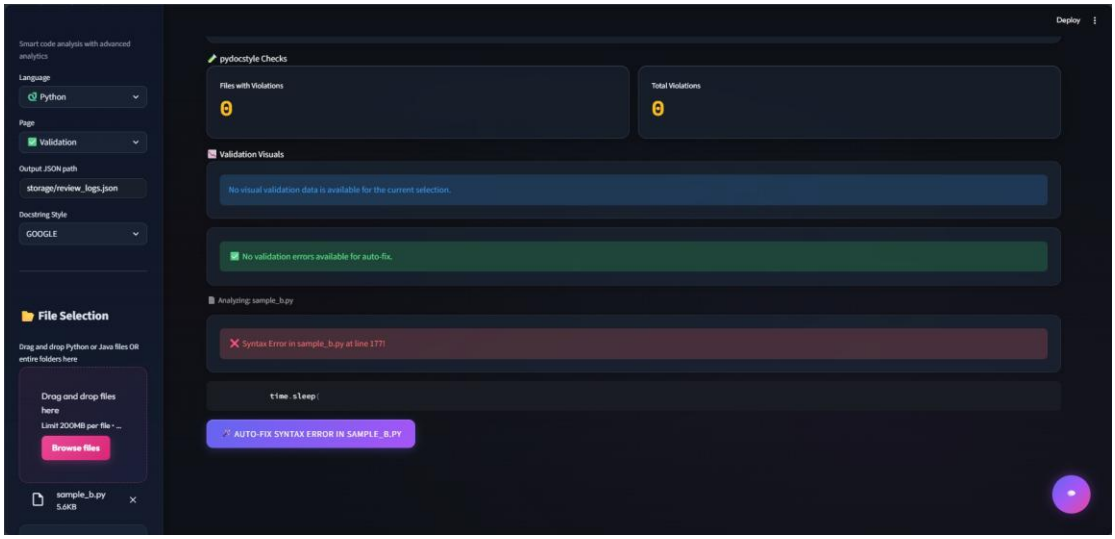


Fig 5.5(a): Syntax Fixer

For Java source code, the system checks the availability and structure of Javadocs along with method and class-level documentation details. It identifies missing Javadocs, incomplete descriptions, and structural issues within Java files to help maintain proper coding and documentation practices.

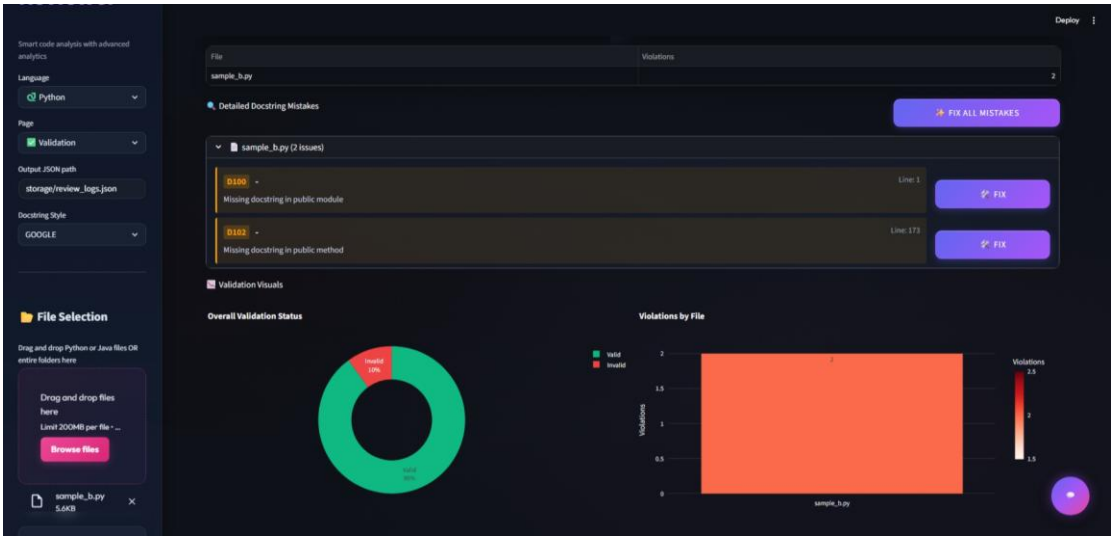


Fig 5.5(b): Validation Page

The interface also supports syntax error detection and correction for both Python and Java programs. When syntax-related problems are found, the system provides corrected suggestions to help users fix the issues more quickly. All validation results, coverage information, and syntax correction outputs are displayed in a clean and organized format, making it easier for users to review and improve their source code.

5.6 AI Optimizer Interface

The AI Optimizer interface helps users improve their source code by providing intelligent suggestions for better coding practices and cleaner program structure. The system analyzes Python functions and Java methods to identify complex logic, repetitive code, unnecessary statements, and areas where the code can be simplified or organized more effectively. These suggestions help improve readability and make the code easier to maintain in the future.

For Python source code, the optimizer studies loops, conditions, variable usage, and function flow to recommend cleaner and more efficient coding approaches. It helps reduce unnecessary complexity while keeping the original functionality of the program unchanged.

For Java source code, the system examines class structures, methods, conditions, and repeated code patterns to generate optimization recommendations. The

suggestions focus on improving code organization, readability, and maintainability for Java applications.

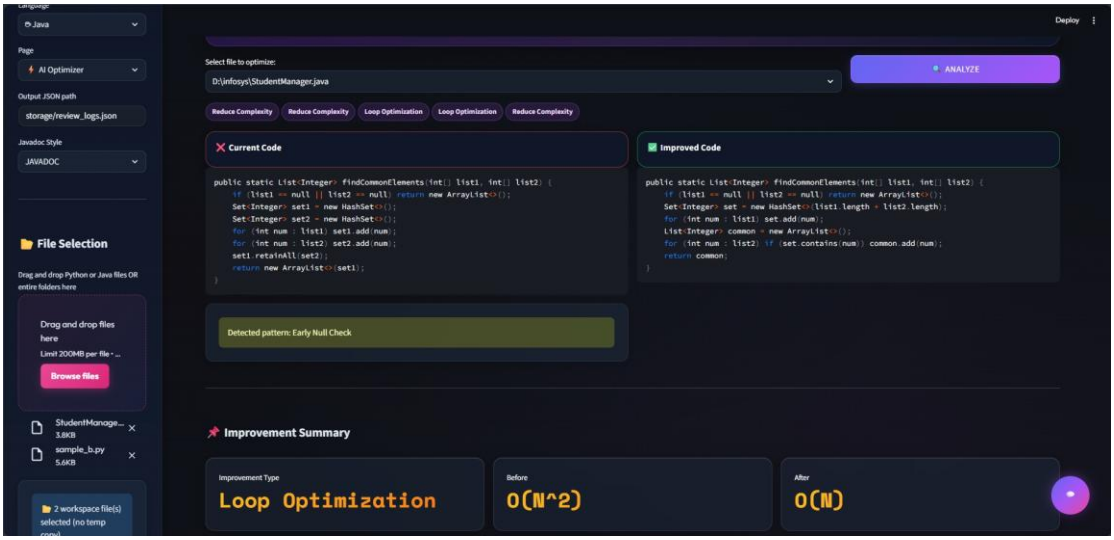


Fig 5.6: AI Optimizer Page

All optimization suggestions are displayed in a clear and structured format so users can review the recommended changes before updating the source code. This interface helps developers create cleaner, more understandable, and better-organized programs for both Python and Java projects.

5.7 AI Assistant Interface

The AI Assistant interface is designed as an interactive chatbot that helps users with Python and Java programming tasks. Users can communicate with the assistant through a chat-based interface to ask questions, understand source code, receive explanations, and get guidance related to different programming concepts. The chatbot provides a simple and user-friendly way to interact with the system while working on software analysis and development tasks.

The assistant is capable of analyzing the currently selected Python or Java source code file and generating responses based on that specific file. Users can ask questions about functions, methods, classes, variables, loops, conditions, exceptions, and overall program behavior. The chatbot studies the uploaded code and provides context-aware answers to help users understand the logic and structure of the program more easily.

For Python source code, the system uses AST-based analysis to examine functions, classes, parameters, decorators, return statements, and control-flow structures before generating responses. For Java source code, the application uses

Regex-based analysis to identify methods, constructors, parameters, access modifiers, and class structures so the chatbot can answer questions related to the selected Java file accurately.

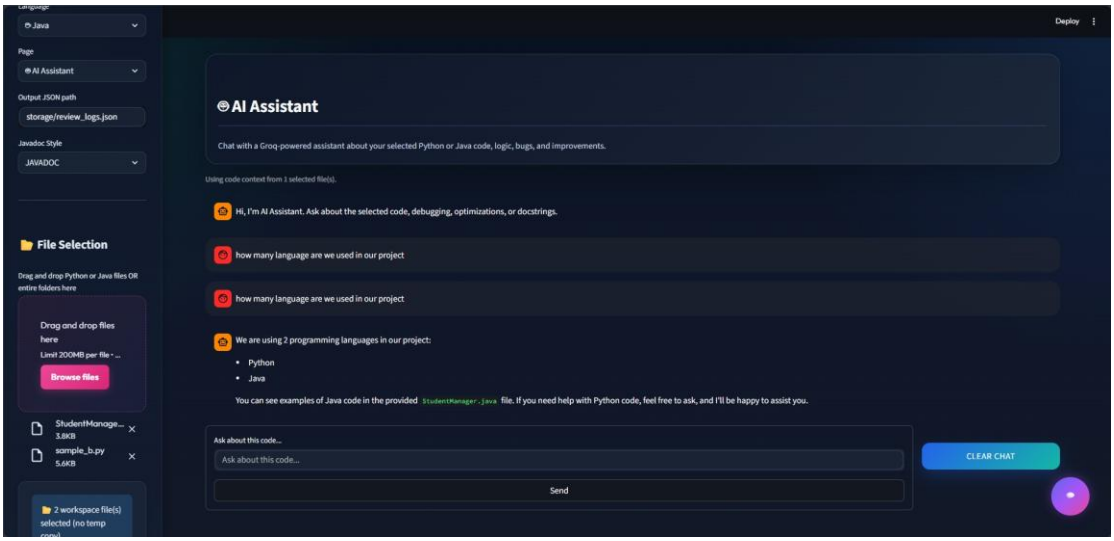


Fig 5.7: AI Assistant

The responses generated by the assistant are displayed in a clean conversational format, making the interaction feel natural and easy to follow. This feature is especially useful for students, developers, and software teams who need quick explanations, debugging assistance, or better understanding of Python and Java projects.

5.8 Dashboard Interface

The Dashboard interface provides a centralized view of software analysis results, project metrics, and visual reports generated by the AI-Powered Code Reviewer and Quality Assistant. It helps users monitor the overall condition of Python and Java projects through interactive analytics, charts, and summarized quality information. The dashboard is designed to present complex software analysis data in a simpler and more understandable format.

The dashboard contains multiple sections such as Key Performance Indicators (KPIs), test results, code quality analysis, function complexity metrics, documentation statistics, and project summaries with smart recommendations. These sections help users quickly understand important project details including documentation coverage, maintainability scores, complexity levels, and overall software quality status.

The interface uses interactive Plotly visualizations including bar charts, scatter plots, histograms, pie charts, and donut charts to display analysis results visually. Features such as hover tooltips, zooming, panning, and chart export options improve interaction

and make data analysis easier for users. The system also supports downloading charts as PNG images and exporting reports in JSON format for future reference.

The dashboard provides detailed metrics such as documentation coverage percentages, maintainability analysis, per-function complexity values, file-level analysis, and lists of highly complex functions that may require optimization or restructuring. Integration with pytest also allows the system to display testing-related results and project accuracy information where applicable.

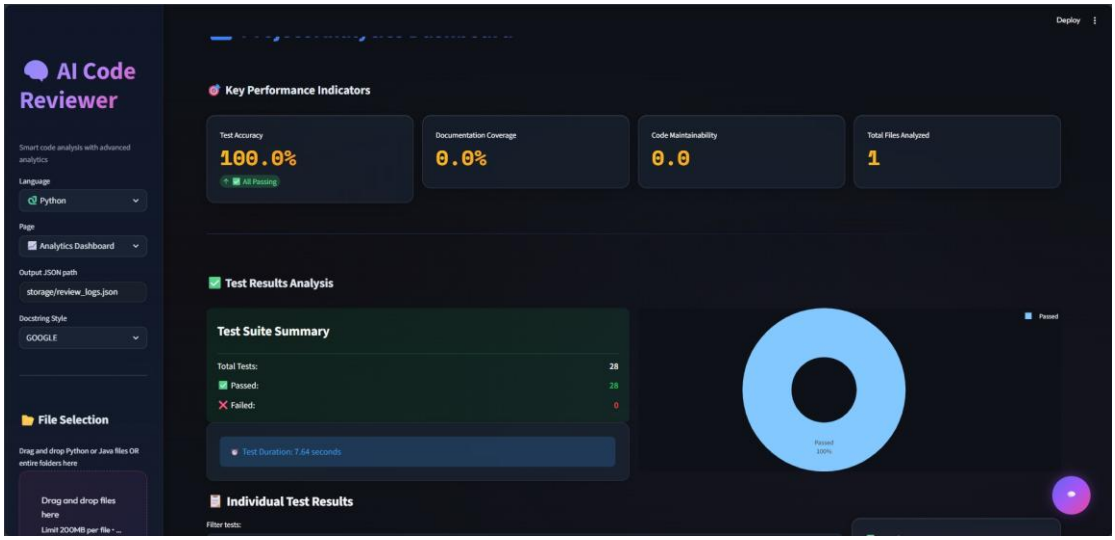


Fig 5.8(a): Dashboard Page

In addition to displaying metrics, the dashboard also generates automated recommendations and insights based on the analysis results. The system highlights areas that may require documentation improvements, refactoring, optimization, or code cleanup and provides actionable suggestions to help improve software quality.

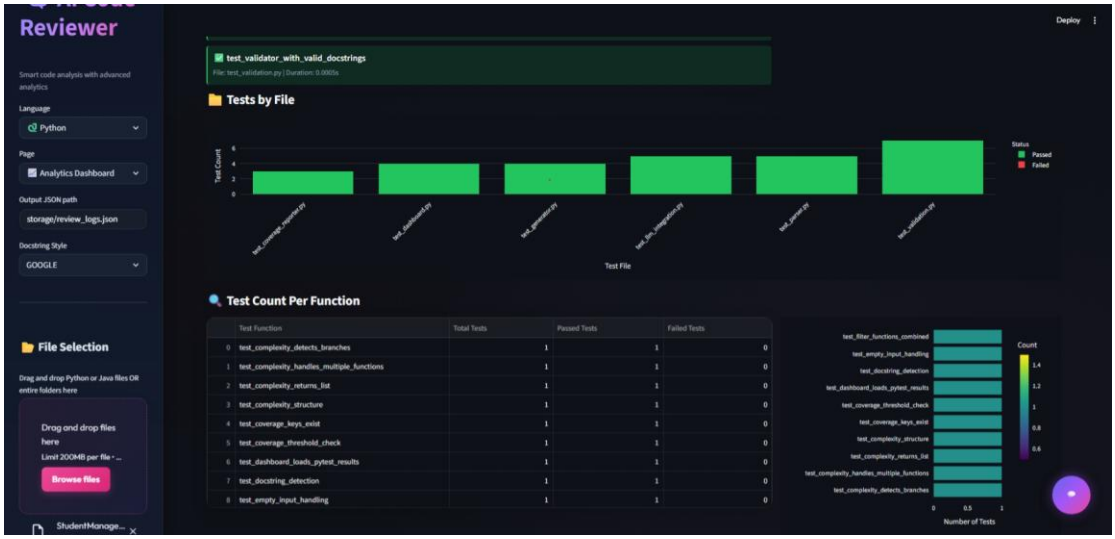


Fig 5.8(b): Testing Result

The interface is designed with a modern and responsive layout that supports different screen sizes ranging from mobile devices to desktop systems. Features such as dark-theme support, color-coded indicators, real-time metric updates, optimized data loading, and caching improve both performance and user experience while interacting with the dashboard.

5.9 Backend Development Implementation

The backend of the AI-Powered Code Reviewer and Quality Assistant is developed using Python to manage the major processing operations performed within the system. It acts as the core layer responsible for handling source code analysis, Artificial Intelligence communication, validation operations, metrics calculation, report generation, and data processing activities. The backend is designed to support both Python and Java source code processing while maintaining smooth communication between different modules of the application.

The implementation follows a modular development approach where individual components perform dedicated tasks while working together to complete the overall workflow. Backend modules manage operations such as source code reading, AST-based analysis for Python files, Regex-based processing for Java files, metadata extraction, AI request handling, documentation generation, validation checking, and software quality evaluation. This separation of functionality improves maintainability and makes future enhancements easier to integrate.

The backend also manages interaction with external technologies and libraries including Groq API, LangChain, Radon, and pydocstyle. These integrations support AI-generated documentation, code explanation, complexity analysis, maintainability evaluation, and validation mechanisms within the application. The backend coordinates information flow between these services and ensures that generated outputs are processed correctly before being displayed to users.

Communication between the frontend interface and processing modules is also controlled through the backend layer. Uploaded files, generated reports, validation results, dashboard analytics, and AI-generated responses move through backend processing mechanisms before being presented within the application interface. This implementation creates an organized processing environment that supports intelligent code review, automated documentation generation, and software quality analysis efficiently.

5.10 Python AST and Java Processing Implementation

Python source code analysis in the AI-Powered Code Reviewer and Quality Assistant is implemented using Python's built-in Abstract Syntax Tree (AST) library. AST-based processing helps the application study the internal organization of Python programs by transforming source code into a structured tree representation. Unlike basic text scanning methods, this approach allows the system to interpret programming elements more accurately and improves the effectiveness of code analysis operations.

Whenever Python files are uploaded, they are transferred to the AST analysis module for processing. The parser examines the source code and extracts important components including functions, classes, parameters, return statements, decorators, loops, conditional blocks, exception-handling structures, and other control-flow elements. Identifying these structures allows the application to understand program organization before performing additional processing tasks.

The extracted details are stored in structured metadata form and shared with different parts of the system. Documentation-related modules use this information to generate Python docstrings automatically, while validation components use it to perform quality checks and documentation analysis. AI-assisted features and explanation modules also rely on the extracted metadata to generate meaningful outputs and improve understanding of the uploaded code.

Using AST-based processing improves source code interpretation because the application works directly with program structure rather than depending entirely on text pattern matching methods. This implementation supports operations such as documentation generation, validation, maintainability evaluation, and software quality assessment while strengthening the AI-assisted analysis capabilities of the application.

Artificial Intelligence integration in the AI-Powered Code Reviewer and Quality Assistant is implemented to support intelligent source code analysis, automated documentation creation, optimization recommendations, and programming assistance for Python and Java applications. The implementation combines Groq API and LangChain technologies to establish a workflow that enables the application to process programming information and generate context-based outputs automatically.

The AI workflow starts after source code analysis and metadata collection are completed. Important information extracted from uploaded files, including function names, parameters, return values, exception details, and control-flow information, is

organized before AI processing begins. The backend module prepares structured prompts using this extracted data so that the language model can interpret the program more accurately and generate better responses.

LangChain is used to manage interactions between backend components and the AI service. It helps organize prompt handling, request management, response processing, and communication flow throughout the application. Groq API is responsible for generating AI-driven outputs by processing the prepared prompts and returning information related to documentation generation, code explanation, optimization guidance, validation support, and programming assistance.

The responses generated by the AI layer undergo additional processing before being presented to users inside the interface. Depending on the selected operation, outputs may include Python docstrings, Java Javadocs, function explanations, optimization suggestions, debugging guidance, and code-related recommendations. Response formatting mechanisms are also applied to improve readability and maintain consistency across generated outputs.

This implementation combines AI capabilities with software analysis methods to create a more intelligent development support environment. Automating activities such as documentation generation, code understanding, and software review reduces repetitive work and assists users in maintaining code quality more efficiently throughout the development process.

5.11 Documentation Generation Implementation

Documentation generation in the AI-Powered Code Reviewer and Quality Assistant automates the creation of structured documentation for Python and Java source code. This feature cuts down on manual documentation tasks and makes things more consistent and clear across software projects. By combining source code analysis with AI processing, the application generates documentation based on the structure and functionality of uploaded programs.

The process starts after the application analyzes the source code and gathers metadata. It extracts information like function names, class details, parameters, return values, exception information, and program structure from the source files. This information is used to generate documentation and helps create outputs that show the program's behavior and purpose.

For Python files, the application produces docstrings using metadata gathered through AST-based processing along with AI-generated content. It supports different documentation styles, such as Google Style, NumPy Style, and reStructuredText (reST). Users can choose formats based on their project needs. The application also creates important sections like parameter descriptions, return information, exception details, and usage examples to enhance code understanding.

For Java programs, the application processes details about classes, methods, constructors, and parameters to generate documentation. Based on this information, it produces Javadoc comments that improve readability and help maintain clear project documentation standards. The implementation also features a preview function. This allows users to review the generated documentation before changing the source files. The automatic insertion support helps add the generated content while keeping the original indentation and formatting. Batch processing capability is included to manage documentation generation across multiple files, which benefits larger projects.

By automating documentation tasks, the implementation reduces repetitive work and makes it easier to maintain software projects. This approach supports clearer documentation practices and makes source code simpler to understand, maintain, and manage over time.

5.12 Validation Engine And Error Handling Implementation

The Validation Engine in the AI-Powered Code Reviewer and Quality Assistant is developed to examine source code quality, documentation standards, and structural consistency in both Python and Java programs. The purpose of this implementation is to identify issues such as missing documentation, formatting mistakes, coding irregularities, and syntax-related problems that may reduce readability or make software maintenance more difficult. Automating these checks reduces the need for extensive manual review and helps maintain better coding practices during development.

The validation workflow starts after source code analysis and metadata extraction are completed. Information collected from source files, including functions, classes, methods, parameters, return values, and documentation details, is passed to validation modules for further examination. The engine studies this information and verifies whether the code and documentation follow expected formatting rules and development standards.

For Python programs, the implementation uses the pydocstyle library to validate docstrings according to PEP 257 documentation guidelines. The system checks for problems such as missing docstrings, incomplete descriptions, formatting mistakes, and documentation inconsistencies. Validation rules related to modules, classes, and functions are examined automatically so that documentation quality can be improved without requiring continuous manual checking. For Java programs, the validation mechanism focuses on documentation completeness and structural organization. The system reviews Javadoc availability, method descriptions, class-level information, and overall code organization. Missing or incomplete documentation sections are highlighted to help maintain better readability and consistency across Java projects.

The engine also performs syntax-related verification and software quality checks by examining source code structure and identifying issues that may influence maintainability or reliability. The generated validation results are organized into reports and displayed through the dashboard interface so users can quickly identify sections that require correction or improvement. By combining automated validation methods with software quality analysis techniques, the implementation helps maintain consistent documentation, encourages cleaner coding practices, and supports better long-term maintainability for both Python and Java software projects.

The Error Handling and Recovery mechanism in the AI-Powered Code Reviewer and Quality Assistant is implemented to maintain system reliability and ensure smooth execution during different software analysis operations. Since the application performs tasks such as source code processing, AI communication, validation, documentation generation, and report preparation, errors can occur during execution. The implementation is designed to detect such situations and handle them in a controlled manner so that the application can continue functioning without affecting the complete workflow.

Exception-handling mechanisms are integrated across backend modules to manage issues that may occur during file uploads, source code reading, metadata extraction, API interactions, validation processes, and report generation activities. When a problem is identified, the system avoids unexpected termination and attempts to continue processing using recovery procedures wherever possible. The application also manages file-related issues such as unsupported file formats, missing files, incorrect source code structures, and encoding problems. Support for encoding formats

like UTF-8, UTF-16, Latin-1, and CP1252 improves compatibility and reduces failures while processing files generated from different development environments.

AI-related operations include additional handling mechanisms for situations such as API connection failures, timeout conditions, unavailable services, or invalid responses. If an AI request cannot be completed successfully, the system informs users through meaningful notifications and fallback responses while preserving previously completed analysis results. The implementation also focuses on user-friendly error reporting by displaying understandable messages instead of complex technical failures. These notifications help users identify issues quickly and improve the overall usability of the application. Combining exception handling, recovery procedures, and controlled failure management helps maintain stable operation and improves reliability for both Python and Java source code analysis workflows.

The Error Handling and Recovery mechanism within the AI-Powered Code Reviewer and Quality Assistant is implemented to improve system reliability, stability, and continuous operation during source code analysis and documentation generation activities. Since the application performs multiple operations such as file processing, source code parsing, Artificial Intelligence communication, validation, report generation, and dashboard visualization, unexpected errors may occur during execution. The error-handling framework is designed to identify these issues, notify users appropriately, and ensure that the application continues functioning without complete system failure.

The implementation begins at the file processing stage, where uploaded Python and Java source code files are validated before analysis. The system checks whether the selected file exists, verifies the file format, and detects encoding types such as UTF-8, UTF-16, Latin-1, and CP1252. If a file cannot be read due to unsupported encoding or corruption, the application catches the exception and displays a user-friendly error message instead of terminating execution. This approach improves usability and prevents unexpected crashes during file uploads. During Python source code analysis, the system utilizes the Abstract Syntax Tree (AST) module to parse program structures. Invalid Python syntax may generate parsing exceptions when the AST parser encounters incomplete statements, incorrect indentation, or malformed code. To handle such situations, exception-handling blocks are implemented around AST processing functions. Whenever a syntax-related issue is detected, the system captures the error

details and presents meaningful feedback to the user while preventing interruption of other application modules.

Java source code processing also includes dedicated error-handling mechanisms. Since Java analysis is performed using Regex-based parsing techniques, improperly structured classes, methods, or unmatched braces may produce extraction errors. The parser validates detected code structures and safely skips unsupported patterns when necessary. This ensures that the analysis process continues even when some sections of the uploaded source code cannot be interpreted correctly.

The Artificial Intelligence integration layer includes additional recovery mechanisms for communication with the Groq API. Network connectivity problems, API rate limits, invalid requests, or service unavailability may affect documentation generation and AI-assisted features. To address these issues, API calls are enclosed within exception-handling blocks that capture connection-related failures and timeout exceptions. When an error occurs, the application displays informative messages and allows users to retry the operation without restarting the system.

The validation and metrics modules also implement protective error handling. Libraries such as pydocstyle and Radon may occasionally encounter unsupported source code structures or analysis failures. To prevent these issues from affecting the complete workflow, exceptions are captured and logged while allowing other validation tasks to continue. This approach ensures that partial analysis results remain available even if one validation component encounters a problem. For monitoring and debugging purposes, the application maintains logging records of significant events, warnings, and errors generated during execution. These logs assist developers in identifying the root causes of failures and improving future versions of the system. Detailed error information is stored internally, while users receive simplified and understandable messages through the interface.

Overall, the Error Handling and Recovery Implementation improves the robustness of the AI-Powered Code Reviewer and Quality Assistant by ensuring that failures are detected, managed, and recovered efficiently. The implementation enhances system reliability, prevents unexpected application termination, and provides a smoother user experience during code analysis, documentation generation, validation, and reporting activities.

CONCLUSION

The AI-Powered Code Reviewer and Quality Assistant was developed to simplify several important activities involved in software development, including code review, documentation, validation, and software quality assessment. By bringing these functionalities together into a single platform, the project provides a practical solution that helps developers analyze source code more efficiently and reduce the amount of manual effort required during development and maintenance. One of the key outcomes of the project is the successful implementation of automated documentation generation for both Python and Java programs. Using extracted source code information and AI-based processing, the application can generate meaningful docstrings and Javadocs automatically. This feature improves code readability and helps maintain proper documentation without requiring developers to write every documentation section manually. Support for multiple documentation formats further increases the usability of the system in different development environments.

The project also demonstrates how traditional code analysis techniques can work together with Artificial Intelligence to provide more useful insights into software quality. Python programs are analyzed using AST-based processing, while Java source code is examined using Regex-based analysis. The extracted information is then used for documentation generation, validation, maintainability evaluation, complexity measurement, and intelligent code assistance. This approach allows the application to provide detailed analysis while maintaining efficient performance. Several AI-assisted features were implemented to improve user experience and code understanding. Functionalities such as Function Explanation, AI Optimizer, Bug Fix Suggestions, Python Assistant, Java Assistant, and Java Validator help users understand program behavior, identify potential issues, and improve code quality. These features are particularly useful when working with large projects or unfamiliar source code where understanding the program structure can be time-consuming.

To support software quality evaluation, the application integrates tools such as pydocstyle and Radon for documentation validation, complexity analysis, and maintainability measurement. The generated metrics help users identify sections of code that may require improvement and provide a clearer understanding of the overall condition of the project. Automated reporting further simplifies the review process and helps maintain organized development records. The user interface developed using

Streamlit, along with Plotly-based visualizations, makes the analysis results easier to interpret. Instead of presenting only textual information, the application displays important metrics through interactive charts and dashboards, allowing users to review software quality data in a more understandable format. Report storage and history tracking features also help users compare analysis results across different stages of development. Developing this project provided valuable practical experience in areas such as source code analysis, AI integration, backend development, documentation standards, API communication, dashboard creation, and software quality assessment. The project offered an opportunity to apply theoretical concepts in a real-world implementation and gain a better understanding of how intelligent tools can support modern software engineering practices.

Although the current version supports Python and Java code analysis, there is scope for future enhancement. Additional programming language support, IDE integration, security analysis, automated test generation, cloud deployment, and advanced AI-assisted features can further improve the capabilities of the system and expand its usefulness in professional software development environments.

In summary, the AI-Powered Code Reviewer and Quality Assistant provides an effective platform for source code analysis, automated documentation, validation, and software quality evaluation. By reducing repetitive tasks and assisting users throughout the development process, the system contributes to better code readability, improved maintainability, and more organized software development practices. The project also demonstrates the growing role of Artificial Intelligence in supporting software engineering activities and provides a strong foundation for future improvements and research in this area.

REFERENCES

[1] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed., New York, NY, USA: McGraw-Hill Education, 2019.

[2] I. Sommerville, *Software Engineering*, 10th ed., Boston, MA, USA: Pearson Education, 2015.

[3] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed., Boston, MA, USA: Addison-Wesley Professional, 2018.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed., Cambridge, MA, USA: MIT Press, 2009.

[5] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Boston, MA, USA: Addison-Wesley, 1994.

[6] B. W. Kernighan and R. Pike, *The Practice of Programming*, Boston, MA, USA: Addison-Wesley Professional, 1999.

[7] M. Lutz, *Learning Python*, 5th ed., Sebastopol, CA, USA: O'Reilly Media, 2013.

[8] H. Schildt, *Java: The Complete Reference*, 11th ed., New York, NY, USA: McGraw-Hill Education, 2018.

[9] T. Mens and T. Tourwé, "A survey of software refactoring," *IEEE Transactions on Software Engineering*, vol. 30, no. 2, pp. 126–139, 2004.

[10] M. Allamanis, E. T. Barr, P. Devanbu, and C. Sutton, "A survey of machine learning for big code and naturalness," *ACM Computing Surveys*, vol. 51, no. 4, pp. 1–37, 2018.

[11] Z. Chen and M. Monperrus, "A literature study of embeddings on source code," *ACM Computing Surveys*, vol. 54, no. 3, pp. 1–38, 2021.

[12] A. Bacchelli and C. Bird, "Expectations, outcomes, and challenges of modern code review," in *Proceedings of the 35th International Conference on Software Engineering*, San Francisco, CA, USA, 2013, pp. 712–721.

[13] F. Rahman, D. Posnett, and P. Devanbu, "Recalling the 'imprecision' of static analysis," in *Proceedings of the ACM SIGSOFT Symposium on the Foundations of Software Engineering*, 2012, pp. 1–11.

[14] M. Greiler, K. Herzig, and J. Czerwonka, "Code ownership and software quality: A replication study," in *Proceedings of the 12th Working Conference on Mining Software Repositories*, Florence, Italy, 2015, pp. 2–12.

[15] T. Zimmermann, N. Nagappan, and A. Zeller, “Predicting bugs from history,” in *Software Evolution*, Berlin, Germany: Springer, 2008, pp. 69–88.
[16] M. Kim, T. Zimmermann, and N. Nagappan, “An empirical study of refactoring challenges and benefits at Microsoft,” *IEEE Transactions on Software Engineering*, vol. 40, no. 7, pp. 633–649, 2014.
[17] S. McConnell, *Code Complete: A Practical Handbook of Software Construction*, 2nd ed., Redmond, WA, USA: Microsoft Press, 2004.
[18] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*, Upper Saddle River, NJ, USA: Prentice Hall, 2008.
[19] P. C. Rigby and C. Bird, “Convergent contemporary software peer review practices,” in *Proceedings of the Joint Meeting of the European Software Engineering Conference and ACM SIGSOFT Symposium on the Foundations of Software Engineering*, Szeged, Hungary, 2013, pp. 202–212.
[20] Y. Liu, C. Wang, and Z. Li, “Automatic code documentation generation using deep learning techniques,” *Journal of Systems and Software*, vol. 167, pp. 110-596, 2020.

DrillBit

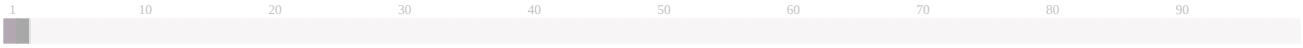
The Report is Generated by DrillBit Plagiarism Detection Software

Submission Information

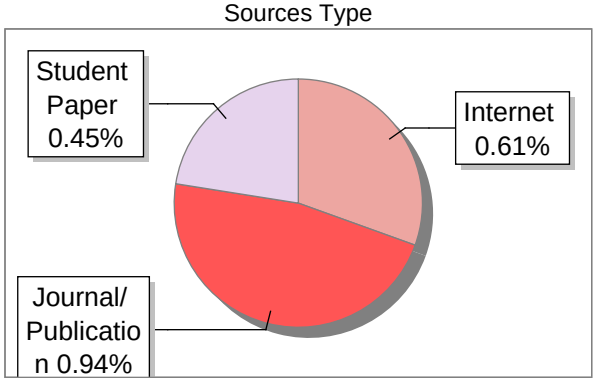
Author Name	Gandhewar Shrikrushna Uday
Title	internship report30.5
Paper/Submission ID	5763447
Submitted by	rajurkar_am@mgmcen.ac.in
Submission Date	2026-05-30
Total Pages, Total Words	59, 16410
Document type	Project Report

Result Information

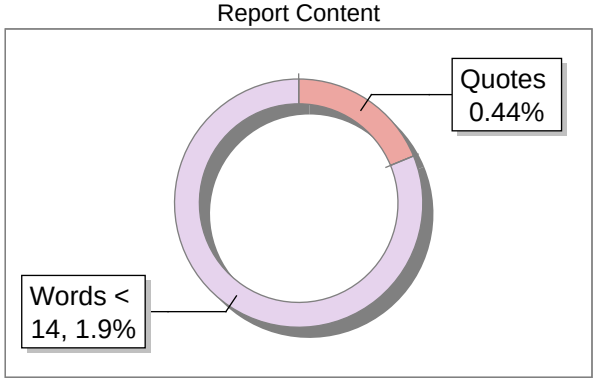
Similarity **2 %**



Sources Type



Report Content



Exclude Information

Quotes	Not Excluded
References/Bibliography	Not Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	**0 %**
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File

DrillBit

DrillBit Similarity Report

2	15	A
SIMILARITY %	MATCHED SOURCES	GRADE

A-Satisfactory (0-10%)
B-Upgrade (11-40%)
C-Poor (41-60%)
D-Unacceptable (61-100%)

LOCATION	MATCHED DOMAIN	%	SOURCE TYPE
3	REPOSITORY - Submitted to Exam section VTU on 2026-02-06 13-00 4422607	<1	Student Paper
4	REPOSITORY - Submitted to Exam section VTU on 2026-02-06 12-48 4422642	<1	Student Paper
9	discovery.ucl.ac.uk	<1	Publication
12	Submitted to Visvesvaraya Technological University, Belagavi	<1	Student Paper
13	acm.org	<1	Internet Data
14	Extending Python for Quantum-classical Computing via Quantum Just-in-time Compi By Thien Nguyen, Alexander J. Mc, Yr-2022,7,27	<1	Publication
16	An analysis of media integration for spatial planning environments by Jones-1994	<1	Publication
17	Characteristics of Water Ingress in Norwegian Subsea Tunnels by Bjr-2014	<1	Publication
18	coek.info	<1	Internet Data
22	translate.google.com	<1	Internet Data
24	ignou.ac.in	<1	Publication
26	pdfs.semanticscholar.org	<1	Publication

27	REPOSITORY - Submitted to Exam section VTU on 2026-02-11 13-47 4400960	<1	Student Paper
30	academicworks.cuny.edu	<1	Internet Data
33	citeseerx.ist.psu.edu	<1	Internet Data